US012388984B2

(12) **United States Patent**
Choi et al.

(10) **Patent No.: US 12,388,984 B2**
(45) **Date of Patent: *Aug. 12, 2025**

(54) **METHOD AND APPARATUS FOR CODING TRANSFORM COEFFICIENT IN VIDEO/IMAGE CODING SYSTEM**

(71) Applicant: **LG Electronics Inc.**, Seoul (KR)

(72) Inventors: **Jungah Choi**, Seoul (KR); **Jaehyun Lim**, Seoul (KR); **Jin Heo**, Seoul (KR); **Sunmi Yoo**, Seoul (KR); **Jangwon Choi**, Seoul (KR); **Seunghwan Kim**, Seoul (KR)

(73) Assignee: **LG Electronics Inc.**, Seoul (KR)

(*) Notice: Subject to any disclaimer, the term of this patent is extended or adjusted under 35 U.S.C. 154(b) by 0 days.

This patent is subject to a terminal disclaimer.

(21) Appl. No.: **18/662,464**

(22) Filed: **May 13, 2024**

(65) **Prior Publication Data**

US 2024/0297984 A1 Sep. 5, 2024

Related U.S. Application Data

(63) Continuation of application No. 17/639,273, filed as application No. PCT/KR2020/011627 on Aug. 31, 2020, now Pat. No. 12,022,061.

(60) Provisional application No. 62/902,977, filed on Sep. 20, 2019, provisional application No. 62/894,773, filed on Aug. 31, 2019.

(51) **Int. Cl.**

H04N 19/105	(2014.01)
H04N 19/169	(2014.01)
H04N 19/176	(2014.01)
H04N 19/18	(2014.01)
H04N 19/70	(2014.01)

(52) **U.S. Cl.**

CPC ***H04N 19/105*** (2014.11); ***H04N 19/176*** (2014.11); ***H04N 19/18*** (2014.11); ***H04N 19/1883*** (2014.11); ***H04N 19/70*** (2014.11)

(58) **Field of Classification Search**

CPC H04N 19/105; H04N 19/176; H04N 19/18; H04N 19/1883; H04N 19/70; H04N 19/91; H04N 19/13; H04N 19/60; H04N 19/124; H04N 19/184

USPC 375/240.02

See application file for complete search history.

(56) **References Cited**

U.S. PATENT DOCUMENTS

10,616,604	B2 *	4/2020	Zhang	H04N 19/593
2016/0286215	A1 *	9/2016	Gamei	H04N 19/107
2020/0077117	A1 *	3/2020	Karczewicz	H04N 19/13

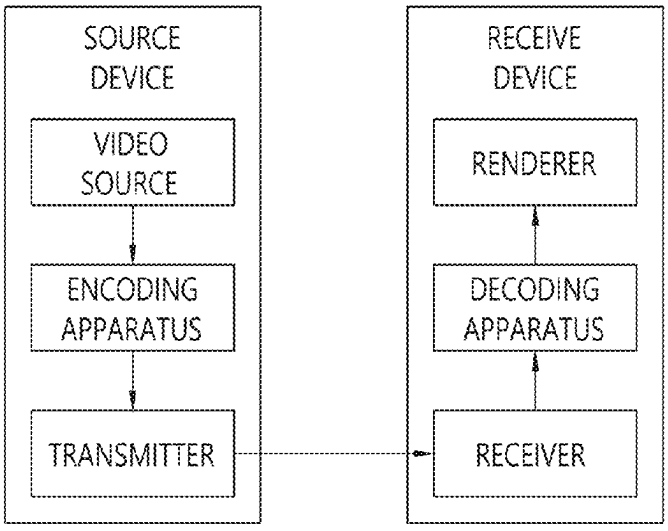
(Continued)

Primary Examiner — Christopher S Kelley
Assistant Examiner — Asmamaw G Tarko
(74) *Attorney, Agent, or Firm* — Fish & Richardson P.C.

(57) **ABSTRACT**

A video decoding method performed by a decoding apparatus according to the present document may comprise the steps of: obtaining, from a bitstream, information indicating a level value of a transform coefficient in a current block; selecting one Rice parameter look-up table for the information indicating the level value of the transform coefficient, from among a plurality of Rice parameter look-up tables; deriving a Rice parameter for the information indicating the level value of the transform coefficient on the basis of the selected Rice parameter look-up table; deriving a bin string for the information indicating the level value of the transform coefficient on the basis of the Rice parameter; and deriving the level value of the transform coefficient on the basis of the bin string.

15 Claims, 14 Drawing Sheets



(56) **References Cited**

U.S. PATENT DOCUMENTS

2020/0267389	A1*	8/2020	George	H04N 19/124
2021/0037261	A1*	2/2021	Hsiang	H04N 19/13
2021/0195197	A1*	6/2021	Kato	H04N 19/13
2022/0286691	A1*	9/2022	Choi	H04N 19/136
2022/0303327	A1*	9/2022	Auyeung	G06F 16/71

* cited by examiner

FIG. 1

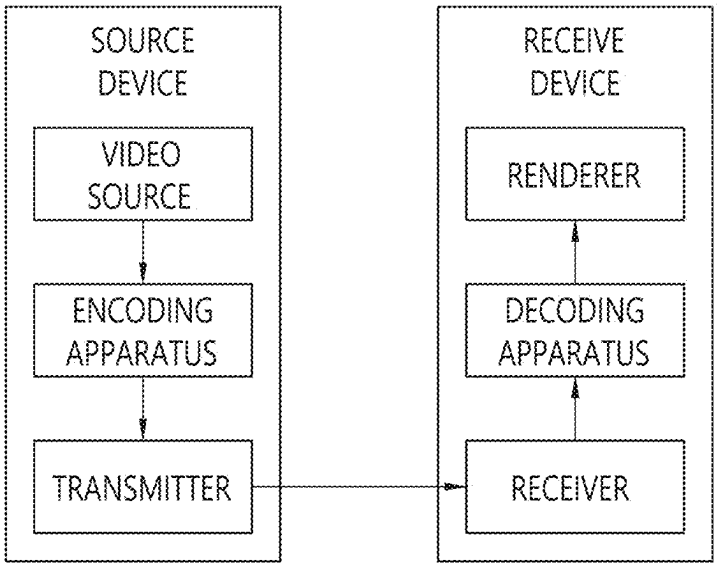

SOURCE DEVICE
VIDEO SOURCE
ENCODING APPARATUS
TRANSMITTER
RECEIVE DEVICE
RENDERER
DECODING APPARATUS
RECEIVER

FIG. 2

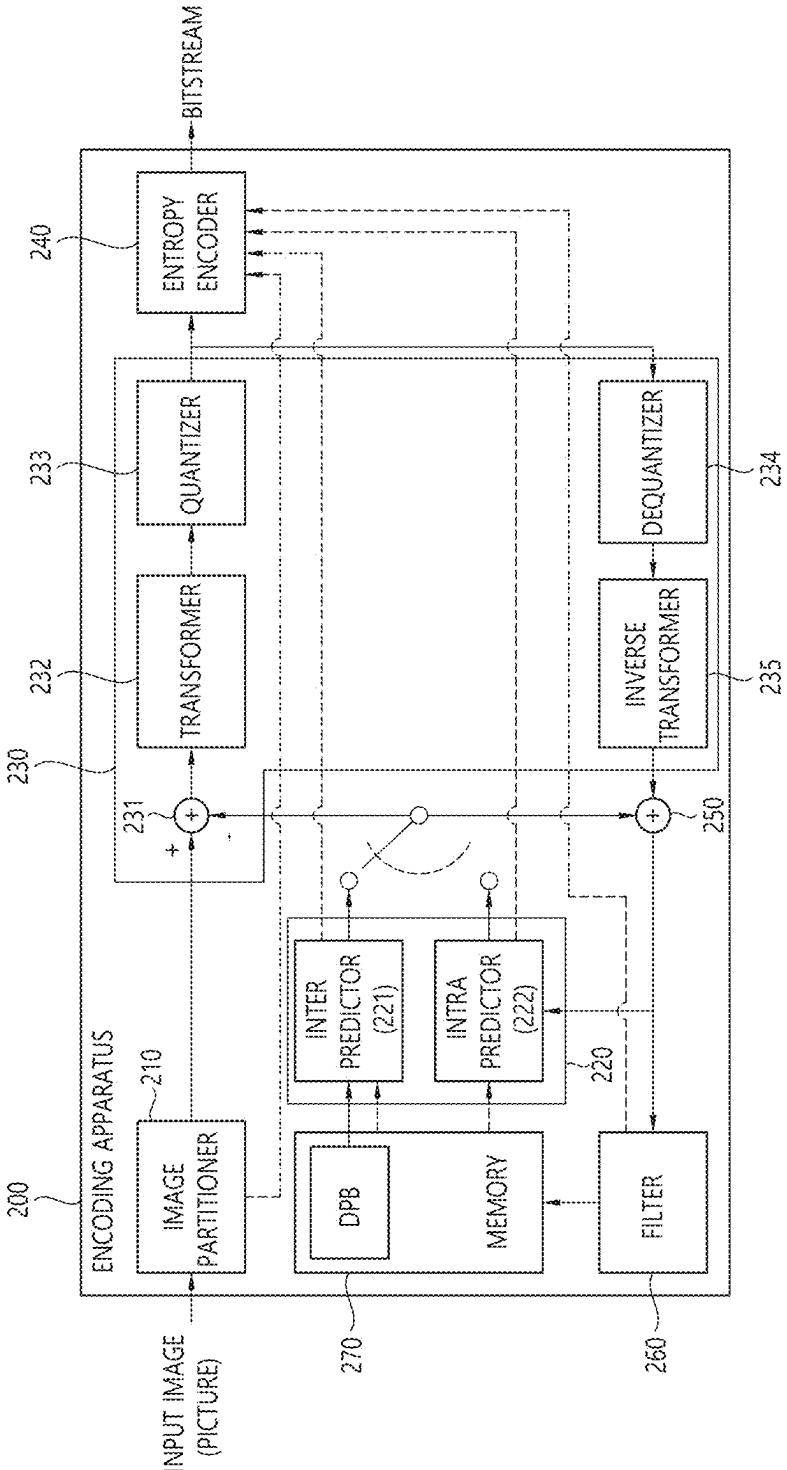

BITSTREAM
ENTROPY ENCODER
240
QUANTIZER
233
TRANSFORMER
232
230
231
+
-
DEQUANTIZER
234
INVERSE TRANSFORMER
235
250
INTER PREDICTOR (221)
INTRA PREDICTOR (222)
220
ENCODING APPARATUS
210
IMAGE PARTITIONER
200
DPB
MEMORY
FILTER
INPUT IMAGE (PICTURE)
270
260

FIG. 3

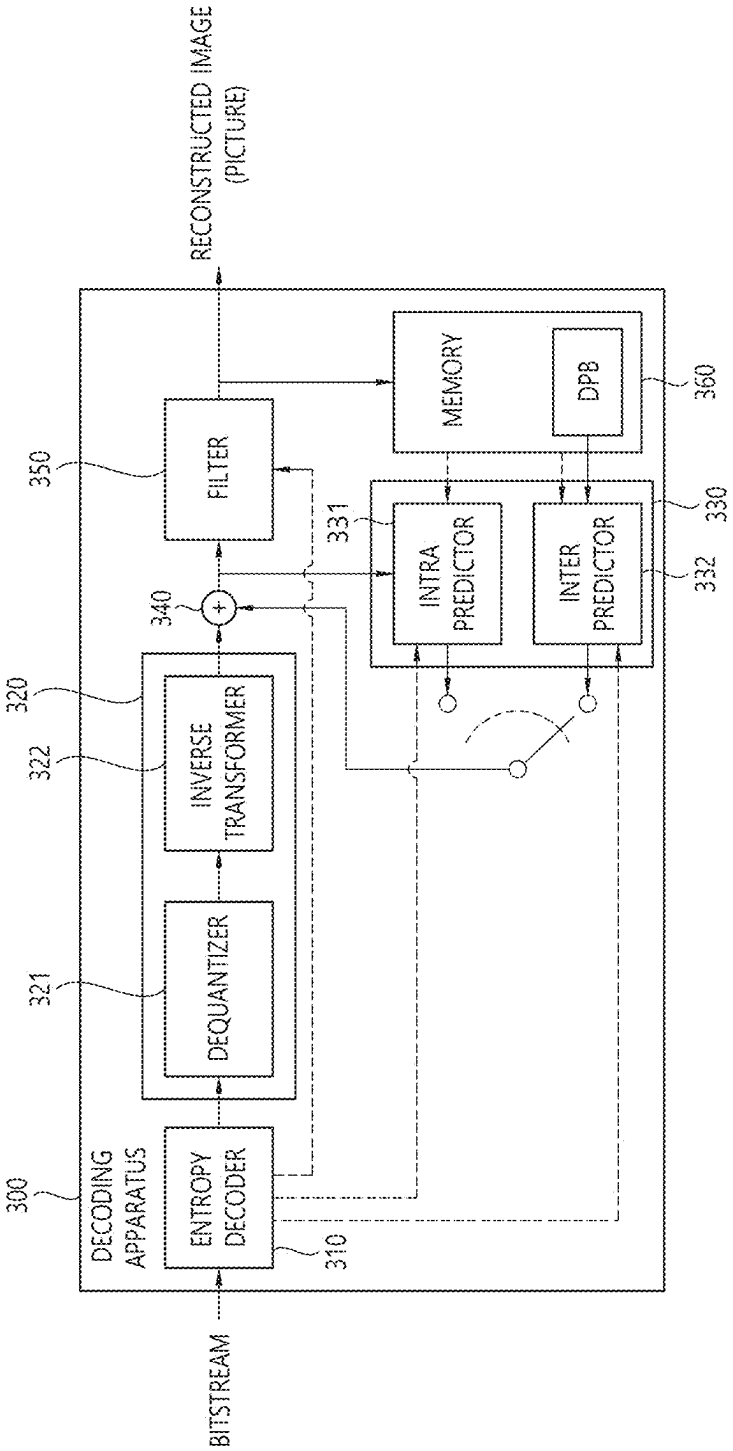

RECONSTRUCTED IMAGE (PICTURE)
BITSTREAM
DECODING APPARATUS
300
ENTROPY DECODER
310
DEQUANTIZER
321
INVERSE TRANSFORMER
322
320
340
FILTER
350
MEMORY
DPB
360
INTRA PREDICTOR
INTER PREDICTOR
331
332
330

FIG. 4

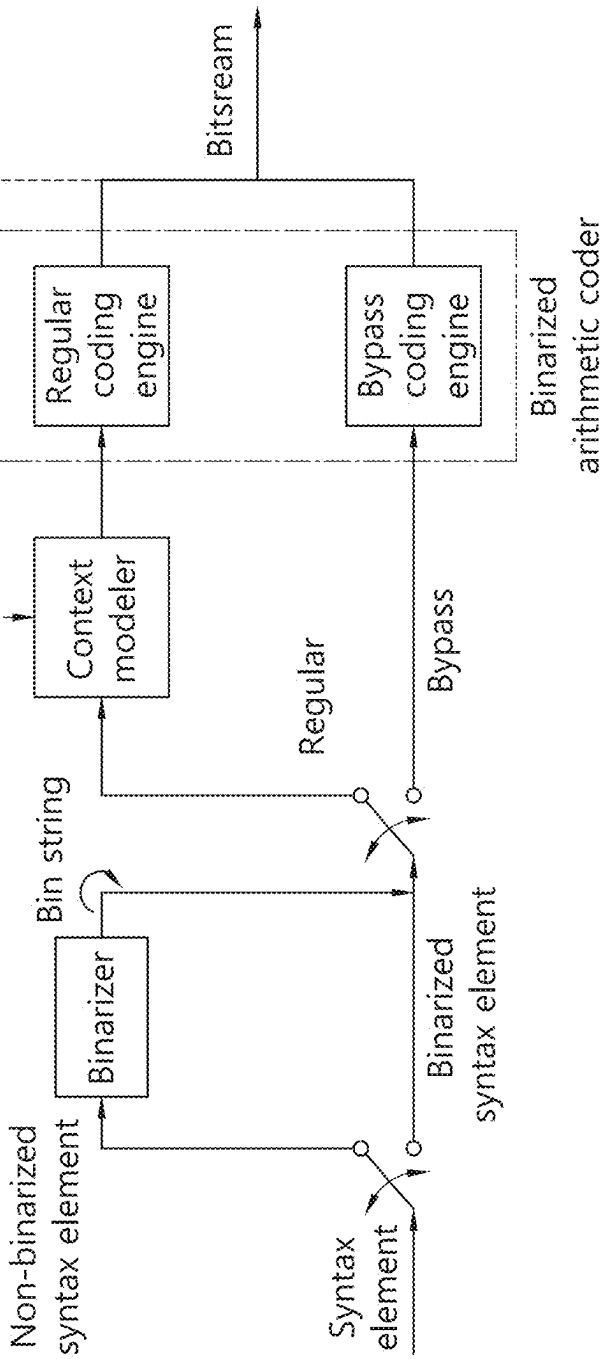

Bitsream
Context model update
Regular coding engine
Bypass coding engine
Binarized arithmetic coder
Context modeler
Regular
Bypass
Bin string
Binarizer
Binarized syntax element
Non-binarized syntax element
Syntax element

FIG. 5

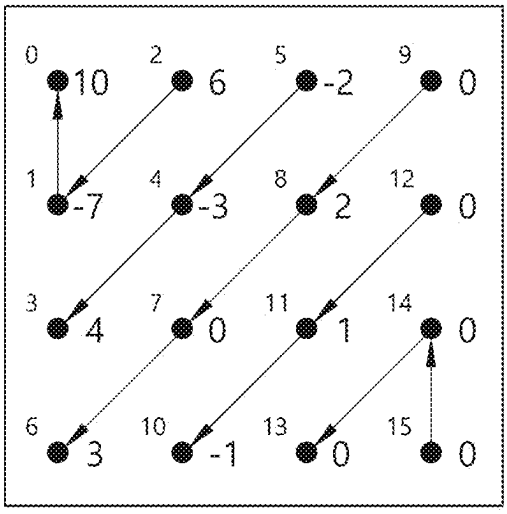


FIG. 6

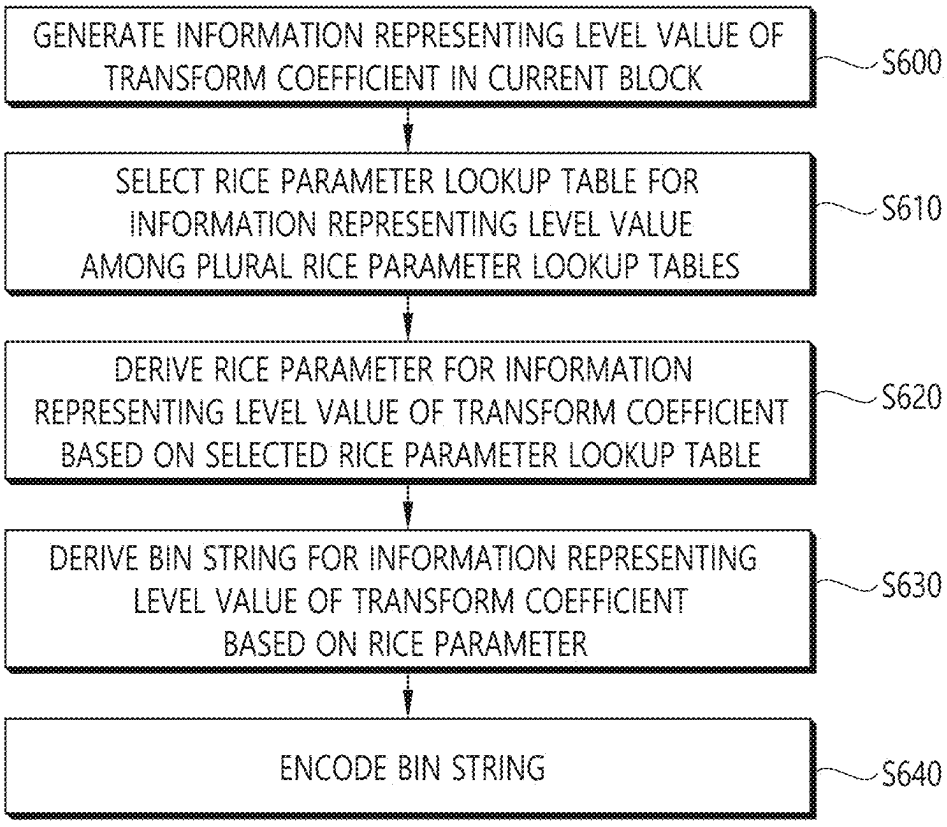

GENERATE INFORMATION REPRESENTING LEVEL VALUE OF TRANSFORM COEFFICIENT IN CURRENT BLOCK
S600
SELECT RICE PARAMETER LOOKUP TABLE FOR INFORMATION REPRESENTING LEVEL VALUE AMONG PLURAL RICE PARAMETER LOOKUP TABLES
S610
DERIVE RICE PARAMETER FOR INFORMATION REPRESENTING LEVEL VALUE OF TRANSFORM COEFFICIENT BASED ON SELECTED RICE PARAMETER LOOKUP TABLE
S620
DERIVE BIN STRING FOR INFORMATION REPRESENTING LEVEL VALUE OF TRANSFORM COEFFICIENT BASED ON RICE PARAMETER
S630
ENCODE BIN STRING
S640

FIG. 7

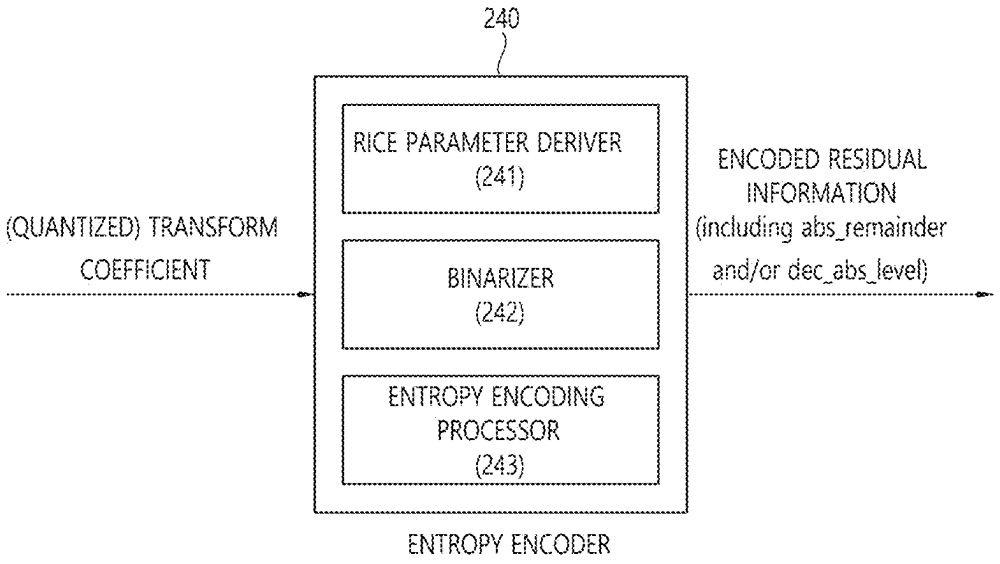

240
RICE PARAMETER DERIVER
(241)
BINARIZER
(242)
ENTROPY ENCODING
PROCESSOR
(243)
ENTROPY ENCODER
(QUANTIZED) TRANSFORM
COEFFICIENT
ENCODED RESIDUAL
INFORMATION
(including abs_remainder
and/or dec_abs_level)

FIG. 8

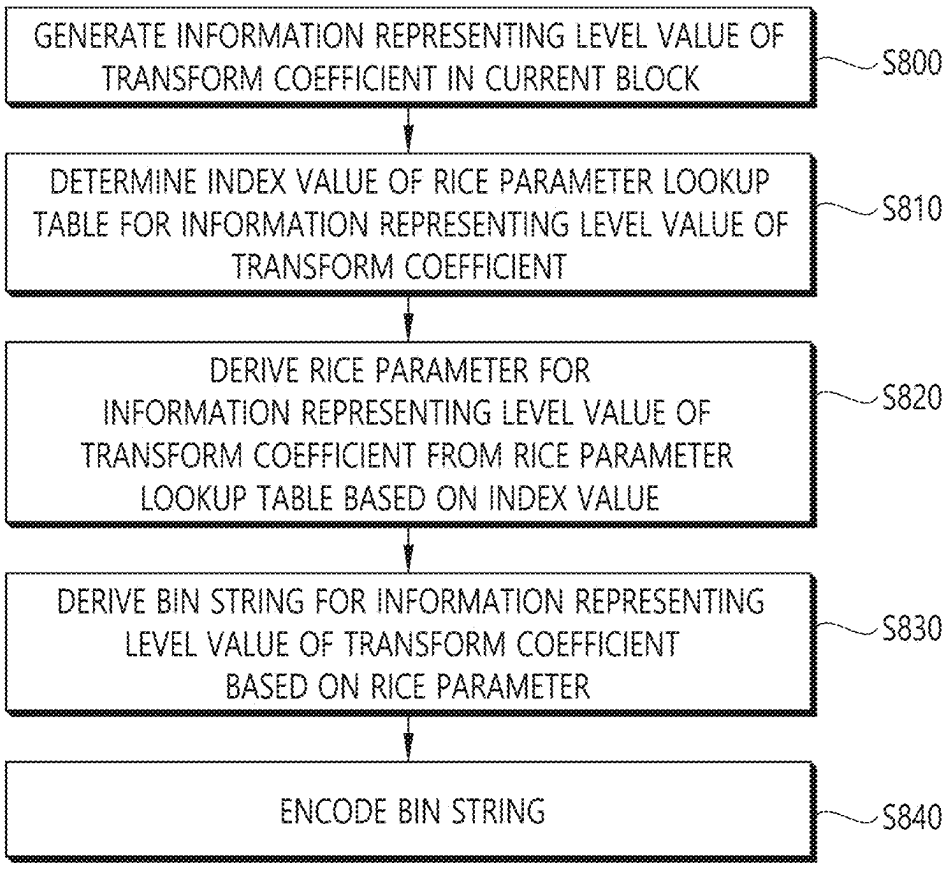

GENERATE INFORMATION REPRESENTING LEVEL VALUE OF TRANSFORM COEFFICIENT IN CURRENT BLOCK
S800
DETERMINE INDEX VALUE OF RICE PARAMETER LOOKUP TABLE FOR INFORMATION REPRESENTING LEVEL VALUE OF TRANSFORM COEFFICIENT
S810
DERIVE RICE PARAMETER FOR INFORMATION REPRESENTING LEVEL VALUE OF TRANSFORM COEFFICIENT FROM RICE PARAMETER LOOKUP TABLE BASED ON INDEX VALUE
S820
DERIVE BIN STRING FOR INFORMATION REPRESENTING LEVEL VALUE OF TRANSFORM COEFFICIENT BASED ON RICE PARAMETER
S830
ENCODE BIN STRING
S840

FIG. 9

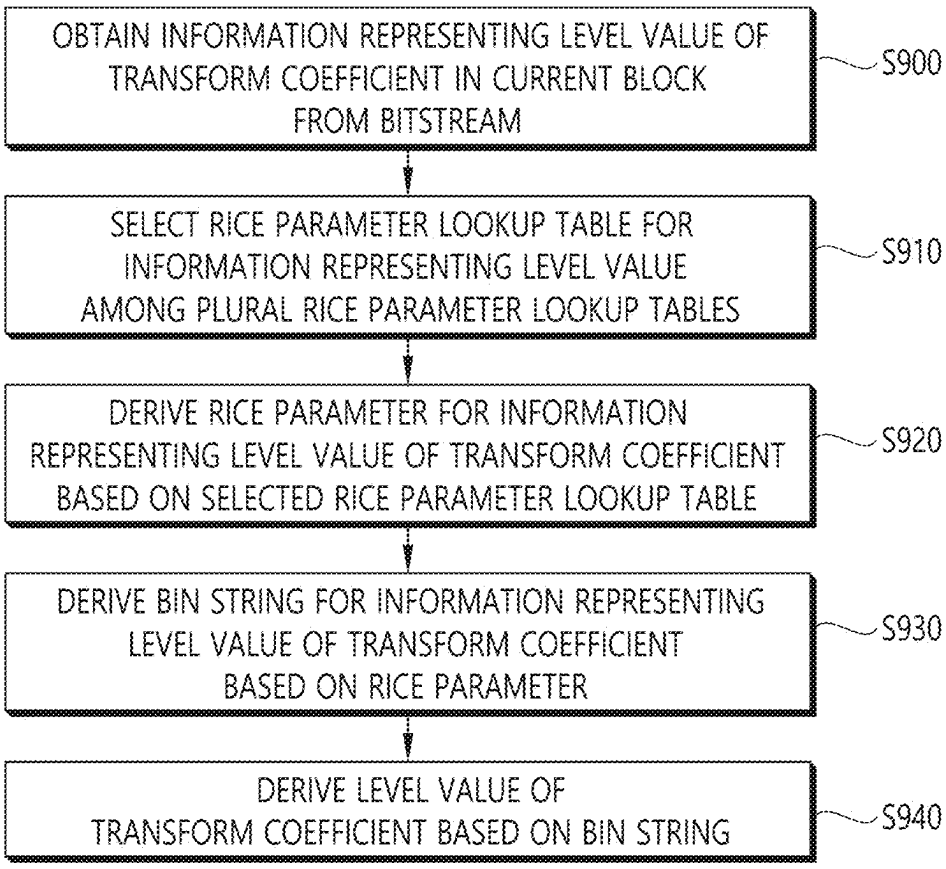

OBTAIN INFORMATION REPRESENTING LEVEL VALUE OF TRANSFORM COEFFICIENT IN CURRENT BLOCK FROM BITSTREAM
S900
SELECT RICE PARAMETER LOOKUP TABLE FOR INFORMATION REPRESENTING LEVEL VALUE AMONG PLURAL RICE PARAMETER LOOKUP TABLES
S910
DERIVE RICE PARAMETER FOR INFORMATION REPRESENTING LEVEL VALUE OF TRANSFORM COEFFICIENT BASED ON SELECTED RICE PARAMETER LOOKUP TABLE
S920
DERIVE BIN STRING FOR INFORMATION REPRESENTING LEVEL VALUE OF TRANSFORM COEFFICIENT BASED ON RICE PARAMETER
S930
DERIVE LEVEL VALUE OF TRANSFORM COEFFICIENT BASED ON BIN STRING
S940

FIG. 10

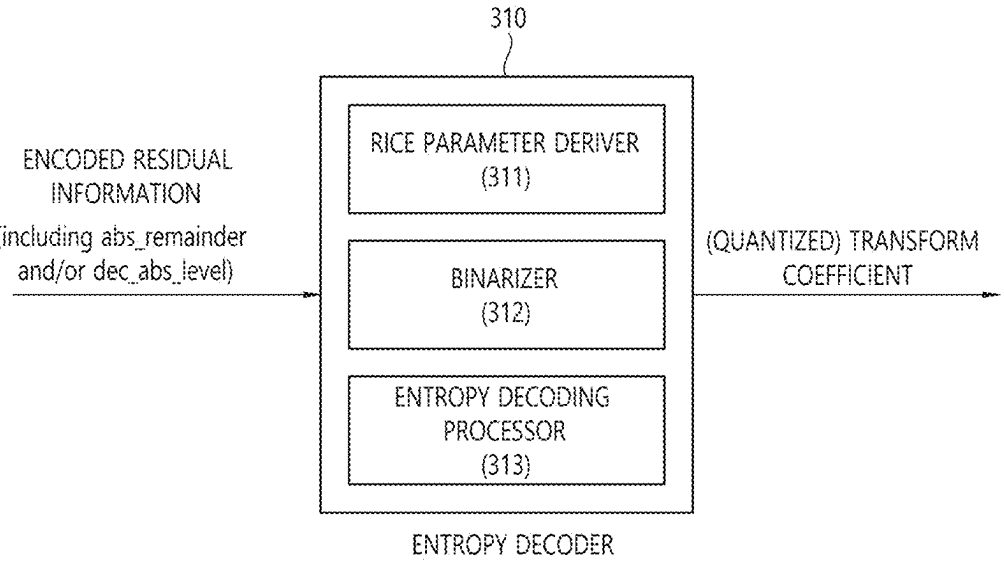

310
ENCODED RESIDUAL INFORMATION
(including abs_remainder and/or dec_abs_level)
RICE PARAMETER DERIVER (311)
BINARIZER (312)
ENTROPY DECODING PROCESSOR (313)
ENTROPY DECODER
(QUANTIZED) TRANSFORM COEFFICIENT

FIG. 11

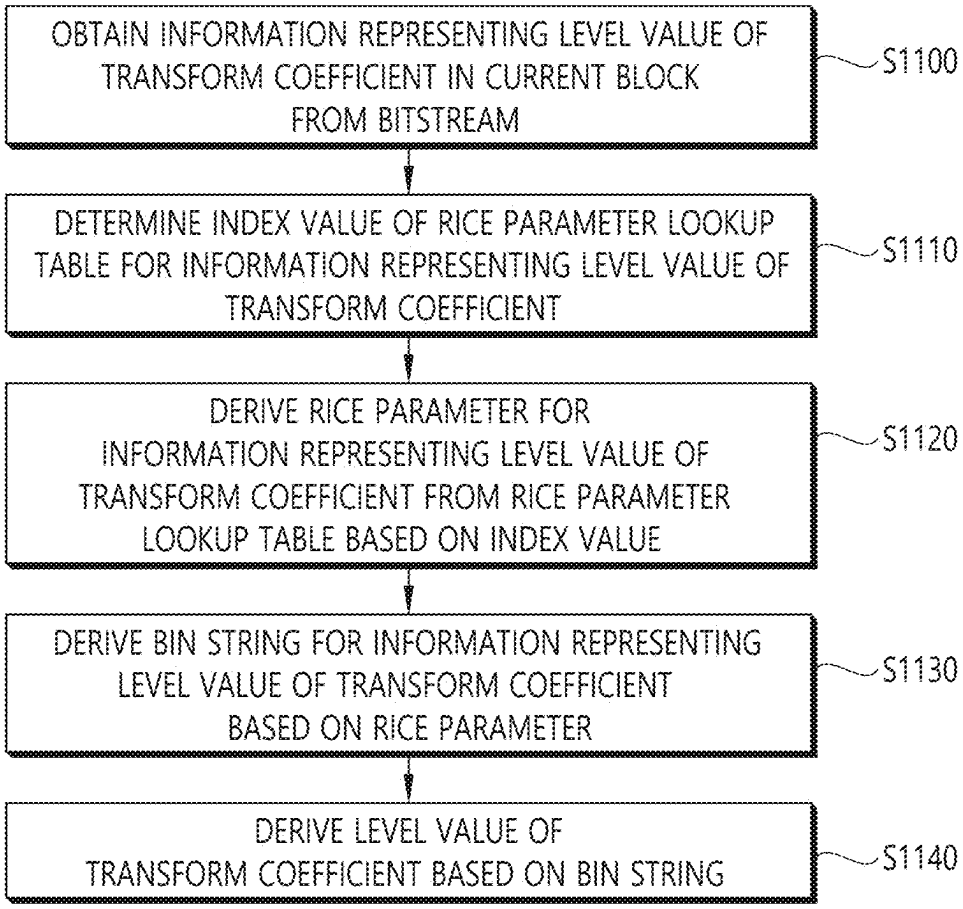

OBTAIN INFORMATION REPRESENTING LEVEL VALUE OF TRANSFORM COEFFICIENT IN CURRENT BLOCK FROM BITSTREAM
S1100
DETERMINE INDEX VALUE OF RICE PARAMETER LOOKUP TABLE FOR INFORMATION REPRESENTING LEVEL VALUE OF TRANSFORM COEFFICIENT
S1110
DERIVE RICE PARAMETER FOR INFORMATION REPRESENTING LEVEL VALUE OF TRANSFORM COEFFICIENT FROM RICE PARAMETER LOOKUP TABLE BASED ON INDEX VALUE
S1120
DERIVE BIN STRING FOR INFORMATION REPRESENTING LEVEL VALUE OF TRANSFORM COEFFICIENT BASED ON RICE PARAMETER
S1130
DERIVE LEVEL VALUE OF TRANSFORM COEFFICIENT BASED ON BIN STRING
S1140

FIG. 12

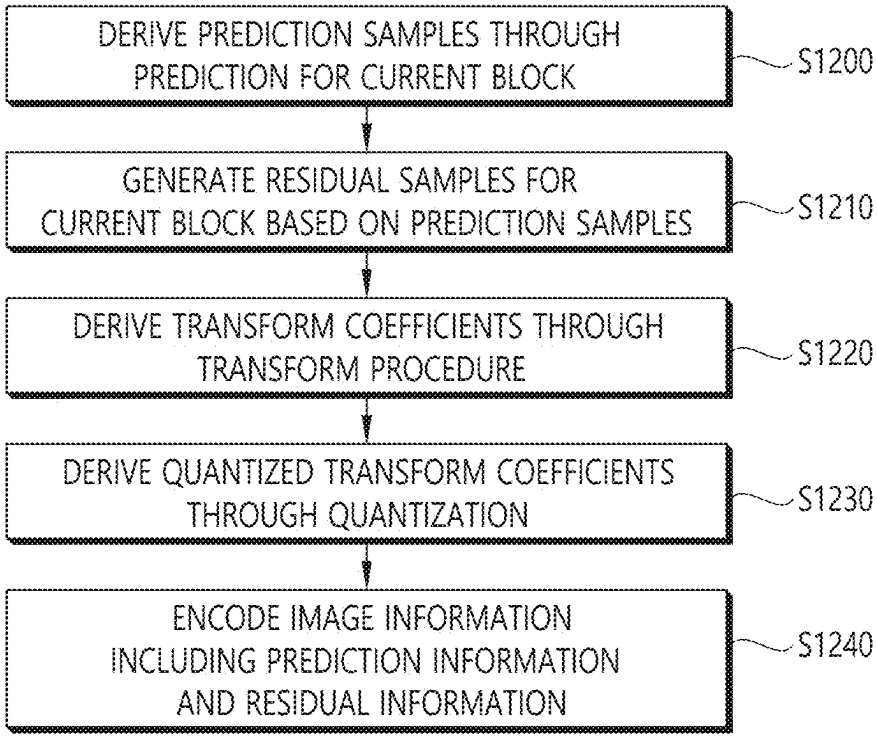

DERIVE PREDICTION SAMPLES THROUGH PREDICTION FOR CURRENT BLOCK
S1200
GENERATE RESIDUAL SAMPLES FOR CURRENT BLOCK BASED ON PREDICTION SAMPLES
S1210
DERIVE TRANSFORM COEFFICIENTS THROUGH TRANSFORM PROCEDURE
S1220
DERIVE QUANTIZED TRANSFORM COEFFICIENTS THROUGH QUANTIZATION
S1230
ENCODE IMAGE INFORMATION INCLUDING PREDICTION INFORMATION AND RESIDUAL INFORMATION
S1240

FIG. 13

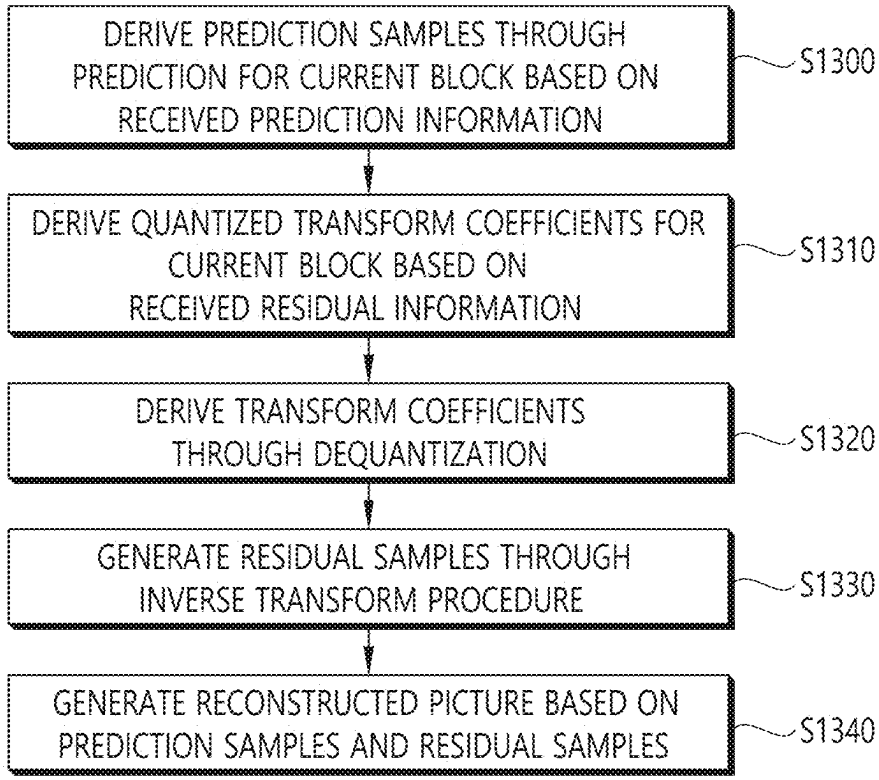
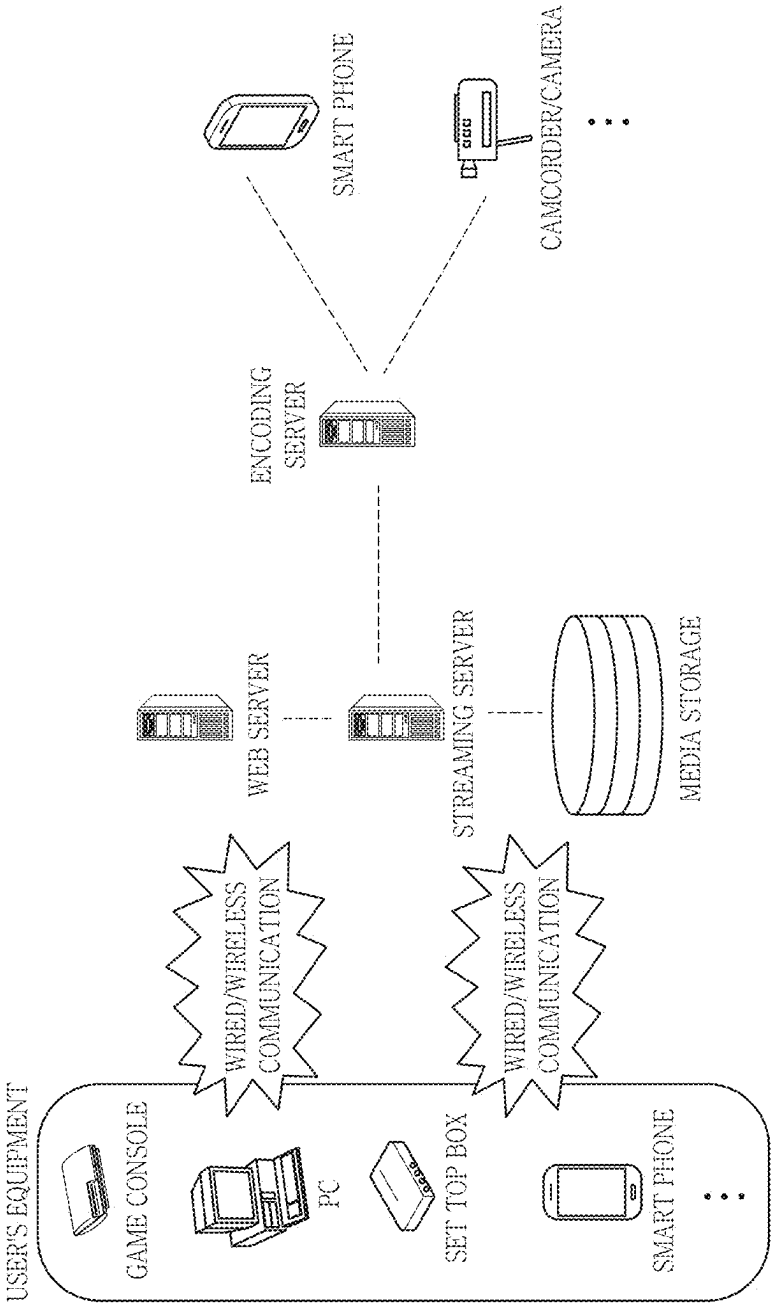

DERIVE PREDICTION SAMPLES THROUGH PREDICTION FOR CURRENT BLOCK BASED ON RECEIVED PREDICTION INFORMATION
S1300
DERIVE QUANTIZED TRANSFORM COEFFICIENTS FOR CURRENT BLOCK BASED ON RECEIVED RESIDUAL INFORMATION
S1310
DERIVE TRANSFORM COEFFICIENTS THROUGH DEQUANTIZATION
S1320
GENERATE RESIDUAL SAMPLES THROUGH INVERSE TRANSFORM PROCEDURE
S1330
GENERATE RECONSTRUCTED PICTURE BASED ON PREDICTION SAMPLES AND RESIDUAL SAMPLES
S1340

FIG. 14


SMART PHONE
CAMCORDER/CAMERA
...
ENCODING SERVER
WEB SERVER
STREAMING SERVER
MEDIA STORAGE
WIRED/WIRELESS COMMUNICATION
WIRED/WIRELESS COMMUNICATION
USER'S EQUIPMENT
GAME CONSOLE
PC
SET TOP BOX
SMART PHONE
...

METHOD AND APPARATUS FOR CODING TRANSFORM COEFFICIENT IN VIDEO/IMAGE CODING SYSTEM

CROSS-REFERENCE TO RELATED APPLICATIONS

This application is a continuation of U.S. application Ser. No. 17/639,273, filed on Feb. 28, 2022, which is a National Stage application under 35 U.S.C. § 371 of International Application No. PCT/KR2020/011627, filed on Aug. 31, 2020, which claims the benefit of U.S. Provisional Application No. 62/894,773, filed on Aug. 31, 2019 and U.S. Provisional Application No. 62/902,977, filed on Sep. 20, 2019. The disclosures of the prior applications are incorporated by reference in their entirety.

BACKGROUND OF THE DISCLOSURE

Field of the Disclosure

The present technology relates to a method and an apparatus for coding a transform coefficient in encoding/decoding a video/image.

Related Art

Recently, the demand for high resolution, high quality image/video such as 4K, 8K or more Ultra High Definition (UHD) image/video is increasing in various fields. As the image/video resolution or quality becomes higher, relatively more amount of information or bits are transmitted than for conventional image/video data. Therefore, if image/video data are transmitted via a medium such as an existing wired/wireless broadband line or stored in a legacy storage medium, costs for transmission and storage are readily increased.

Moreover, interests and demand are growing for virtual reality (VR) and artificial reality (AR) contents, and immersive media such as hologram; and broadcasting of images/videos exhibiting image/video characteristics different from those of an actual image/video, such as game images/videos, are also growing.

Therefore, a highly efficient image/video compression technique is required to effectively compress and transmit, store, or play high resolution, high quality images/videos showing various characteristics as described above.

SUMMARY

A technical subject of the present document is to provide a method and an apparatus for enhancing video/image coding efficiency.

Another technical subject of the present document is to provide a method and an apparatus for improving a residual coding efficiency.

Still another technical subject of the present document is to provide a method and an apparatus for improving coding performance of level coding for a transform coefficient in residual coding.

According to an embodiment of the present document, a video decoding method performed by a decoding apparatus may include: obtaining information representing a level value of a transform coefficient in a current block from a bitstream; selecting any one of a plurality of rice parameter lookup tables with respect to the information representing the level value of the transform coefficient; deriving a rice parameter for the information representing the level value of the transform coefficient based on the selected rice parameter lookup table; deriving a bin string for the information representing the level value of the transform coefficient based on the rice parameter; and deriving the level value of the transform coefficient based on the bin string.

According to another embodiment of the present document, a video decoding method performed by a decoding apparatus may include: obtaining information representing a level value of a transform coefficient in a current block from a bitstream; determining an index value of a rice parameter lookup table for the information representing the level value of the transform coefficient; deriving a rice parameter for the information representing the level value of the transform coefficient from the rice parameter lookup table based on the index value; deriving a bin string for the information representing the level value of the transform coefficient based on the rice parameter; and deriving the level value of the transform coefficient based on the bin string.

According to an embodiment of the present document, the overall video/image compression efficiency can be improved.

According to an embodiment of the present document, the residual coding efficiency can be improved.

According to an embodiment of the present document, the coding performance of the level coding for the transform coefficient in the residual coding can be improved.

According to an embodiment of the present document, in case that a low level value and a high level value of the transform coefficient are mixed, a higher performance can be provided in lossless or high bit rate environment (low QP) having a relatively high level value.

BRIEF DESCRIPTION OF THE DRAWINGS

FIG. **1** schematically illustrates an example of a video/image coding system to which embodiments of the present document are applicable.

FIG. **2** is a schematic diagram illustrating a configuration of a video/image encoding apparatus to which the embodiments of the present document are applicable.

FIG. **3** is a schematic diagram illustrating a configuration of a video/image decoding apparatus to which the embodiments of the present document are applicable.

FIG. **4** exemplarily illustrates context-adaptive binary arithmetic coding (CABAC) for encoding a syntax element.

FIG. **5** exemplarily illustrates transform coefficients in a 4×4 block.

FIGS. **6** and **7** schematically illustrate an example of an entropy encoding method and related components according to an embodiment of the present document.

FIG. **8** schematically illustrates an example of an entropy encoding method according to another embodiment of the present document.

FIGS. **9** and **10** schematically illustrate an example of an entropy decoding method and related components according to an embodiment of the present document.

FIG. **11** schematically illustrates an example of an entropy decoding method according to another embodiment of the present document.

FIG. **12** illustrates a video/image encoding method according to an embodiment of the present document.

FIG. **13** illustrates a video/image decoding method according to an embodiment of the present document.

FIG. **14** illustrates an example of a content streaming system to which embodiments disclosed in the present document are applicable.

DESCRIPTION OF EXEMPLARY EMBODIMENTS

The disclosure of the present document may be modified in various forms, and specific embodiments thereof will be described and illustrated in the drawings. The terms used in the present document are used to merely describe specific embodiments, but are not intended to limit the disclosed method in the present document. An expression of a singular number includes an expression of 'at least one', so long as it is clearly read differently. The terms such as "include" and "have" are intended to indicate that features, numbers, steps, operations, elements, components, or combinations thereof used in the document exist and it should be thus understood that the possibility of existence or addition of one or more different features, numbers, steps, operations, elements, components, or combinations thereof is not excluded.

In addition, each configuration of the drawings described in the present document is an independent illustration for explaining functions as features that are different from each other, and does not mean that each configuration is implemented by mutually different hardware or different software. For example, two or more of the configurations may be combined to form one configuration, and one configuration may also be divided into multiple configurations. Without departing from the gist of the disclosed method of the present document, embodiments in which configurations are combined and/or separated are included in the scope of the disclosure of the present document.

Hereinafter, embodiments of the present document will be described in detail with reference to the accompanying drawings. In addition, like reference numerals are used to indicate like elements throughout the drawings, and the same descriptions on the like elements may be omitted.

FIG. **1** illustrates an example of a video/image coding system to which the embodiments of the present document may be applied.

Referring to FIG. **1**, a video/image coding system may include a first device (a source device) and a second device (a reception device). The source device may transmit encoded video/image information or data to the reception device through a digital storage medium or network in the form of a file or streaming.

The source device may include a video source, an encoding apparatus, and a transmitter. The receiving device may include a receiver, a decoding apparatus, and a renderer. The encoding apparatus may be called a video/image encoding apparatus, and the decoding apparatus may be called a video/image decoding apparatus. The transmitter may be included in the encoding apparatus. The receiver may be included in the decoding apparatus. The renderer may include a display, and the display may be configured as a separate device or an external component.

The video source may acquire video/image through a process of capturing, synthesizing, or generating the video/image. The video source may include a video/image capture device and/or a video/image generating device. The video/image capture device may include, for example, one or more cameras, video/image archives including previously captured video/images, and the like. The video/image generating device may include, for example, computers, tablets and smartphones, and may (electronically) generate video/images. For example, a virtual video/image may be generated through a computer or the like. In this case, the video/image capturing process may be replaced by a process of generating related data.

The encoding apparatus may encode input video/image. The encoding apparatus may perform a series of procedures such as prediction, transform, and quantization for compaction and coding efficiency. The encoded data (encoded video/image information) may be output in the form of a bitstream.

The transmitter may transmit the encoded image/image information or data output in the form of a bitstream to the receiver of the receiving device through a digital storage medium or a network in the form of a file or streaming. The digital storage medium may include various storage mediums such as USB, SD, CD, DVD, Blu-ray, HDD, SSD, and the like. The transmitter may include an element for generating a media file through a predetermined file format and may include an element for transmission through a broadcast/communication network. The receiver may receive/extract the bitstream and transmit the received bitstream to the decoding apparatus.

The decoding apparatus may decode the video/image by performing a series of procedures such as dequantization, inverse transform, and prediction corresponding to the operation of the encoding apparatus.

The renderer may render the decoded video/image. The rendered video/image may be displayed through the display.

The present document relates to video/image coding. For example, a method/embodiment disclosed in the present document may be applied to a method disclosed in a versatile video coding (VVC) standard. In addition, the method/embodiment disclosed in the present document may be applied to a method disclosed in an essential video coding (EVC) standard, AOMedia Video 1 (AV1) standard, 2nd generation of audio video coding standard (AVS2), or a next-generation video/image coding standard (e.g., H.267, H.268, etc.).

Various embodiments related to video/image coding are presented in the present document, and the embodiments may be combined with each other unless otherwise stated.

In the present document, a video may refer to a series of images over time. A picture generally refers to the unit representing one image at a particular time frame, and a slice/tile refers to the unit constituting a part of the picture in terms of coding. A slice/tile may include one or more coding tree units (CTUs). One picture may consist of one or more slices/tiles. One picture may consist of one or more tile groups. One tile group may include one or more tiles. A brick may represent a rectangular region of CTU rows within a tile in a picture). A tile may be partitioned into multiple bricks, each of which consisting of one or more CTU rows within the tile. A tile that is not partitioned into multiple bricks may be also referred to as a brick. A brick scan is a specific sequential ordering of CTUs partitioning a picture in which the CTUs are ordered consecutively in CTU raster scan in a brick, bricks within a tile are ordered consecutively in a raster scan of the bricks of the tile, and tiles in a picture are ordered consecutively in a raster scan of the tiles of the picture. A tile is a rectangular region of CTUs within a particular tile column and a particular tile row in a picture. The tile column is a rectangular region of CTUs having a height equal to the height of the picture and a width specified by syntax elements in the picture parameter set. The tile row is a rectangular region of CTUs having a height specified by syntax elements in the picture parameter set and a width equal to the width of the picture). A tile scan is a specific sequential ordering of CTUs partitioning a picture in

which the CTUs are ordered consecutively in CTU raster scan in a tile whereas tiles in a picture are ordered consecutively in a raster scan of the tiles of the picture. A slice includes an integer number of bricks of a picture that may be exclusively contained in a single NAL unit. A slice may consist of either a number of complete tiles or only a consecutive sequence of complete bricks of one tile. In the present document, tile group and slice may be used interchangeably. For example, in the present document, a tile group/tile group header may be referred to as a slice/slice header.

A pixel or a pel may mean a smallest unit constituting one picture (or image). Also, 'sample' may be used as a term corresponding to a pixel. A sample may generally represent a pixel or a value of a pixel, and may represent only a pixel/pixel value of a luma component or only a pixel/pixel value of a chroma component.

A unit may represent a basic unit of image processing. The unit may include at least one of a specific region of the picture and information related to the region. One unit may include one luma block and two chroma (ex. Cb, cr) blocks. The unit may be used interchangeably with terms such as block or area in some cases. In a general case, an M×N block may include samples (or sample arrays) or a set (or array) of transform coefficients of M columns and N rows. Alternatively, the sample may mean a pixel value in the spatial domain, and when such a pixel value is transformed to the frequency domain, it may mean a transform coefficient in the frequency domain.

In the present document, the term "/" and "," should be interpreted to indicate "and/or." For instance, the expression "A/B" may mean "A and/or B." Further, "A, B" may mean "A and/or B." Further, "A/B/C" may mean "at least one of A, B, and/or C." Also, "A/B/C" may mean "at least one of A, B, and/or C."

Further, in the document, the term "or" should be interpreted to indicate "and/or." For instance, the expression "A or B" may comprise 1) only A, 2) only B, and/or 3) both A and B. In other words, the term "or" in the present document should be interpreted to indicate "additionally or alternatively."

Further, the parentheses used in the present document may mean "for example". Specifically, in case that "prediction (intra prediction)" is expressed, it may be indicated that "intra prediction" is proposed as an example of "prediction". In other words, the term "prediction" in the present document is not limited to "intra prediction", and "intra prediction" is proposed as an example of "prediction". Further, even in case that "prediction (i.e., intra prediction)" is expressed, it may be indicated that "intra prediction" is proposed as an example of "prediction".

In the present document, technical features individually explained in one drawing may be individually implemented or simultaneously implemented.

FIG. **2** is a diagram schematically illustrating the configuration of a video/image encoding apparatus to which the embodiments of the present document may be applied. Hereinafter, what is referred to as the video encoding apparatus may include an image encoding apparatus.

Referring to FIG. **2**, the encoding apparatus **200** may include and be configured with an image partitioner **210**, a predictor **220**, a residual processor **230**, an entropy encoder **240**, an adder **250**, a filter **260**, and a memory **270**. The predictor **220** may include an inter predictor **221** and an intra predictor **222**. The residual processor **230** may include a transformer **232**, a quantizer **233**, a dequantizer **234**, and an inverse transformer **235**. The residual processor **230** may further include a subtractor **231**. The adder **250** may be called a reconstructor or reconstructed block generator. The image partitioner **210**, the predictor **220**, the residual processor **230**, the entropy encoder **240**, the adder **250**, and the filter **260**, which have been described above, may be configured by one or more hardware components (e.g., encoder chipsets or processors) according to an embodiment. In addition, the memory **270** may include a decoded picture buffer (DPB), and may also be configured by a digital storage medium. The hardware component may further include the memory **270** as an internal/external component.

The image partitioner **210** may split an input image (or, picture, frame) input to the encoding apparatus **200** into one or more processing units. As an example, the processing unit may be called a coding unit (CU). In this case, the coding unit may be recursively split according to a Quad-tree binary-tree ternary-tree (QTBTTT) structure from a coding tree unit (CTU) or the largest coding unit (LCU). For example, one coding unit may be split into a plurality of coding units of a deeper depth based on a quad-tree structure, a binary-tree structure, and/or a ternary-tree structure. In this case, for example, the quad-tree structure is first applied and the binary-tree structure and/or the ternary-tree structure may be later applied. Alternatively, the binary-tree structure may also be first applied. A coding procedure according to the present document may be performed based on a final coding unit which is not split any more. In this case, based on coding efficiency according to image characteristics or the like, the maximum coding unit may be directly used as the final coding unit, or as necessary, the coding unit may be recursively split into coding units of a deeper depth, such that a coding unit having an optimal size may be used as the final coding unit. Here, the coding procedure may include a procedure such as prediction, transform, and reconstruction to be described later. As another example, the processing unit may further include a prediction unit (PU) or a transform unit (TU). In this case, each of the prediction unit and the transform unit may be split or partitioned from the aforementioned final coding unit. The prediction unit may be a unit of sample prediction, and the transform unit may be a unit for inducing a transform coefficient and/or a unit for inducing a residual signal from the transform coefficient.

The unit may be interchangeably used with the term such as a block or an area in some cases. Generally, an M×N block may represent samples composed of M columns and N rows or a group of transform coefficients. The sample may generally represent a pixel or a value of the pixel, and may also represent only the pixel/pixel value of a luma component, and also represent only the pixel/pixel value of a chroma component. The sample may be used as the term corresponding to a pixel or a pel configuring one picture (or image).

The encoding apparatus **200** may subtract the prediction signal (predicted block, prediction sample array) output from the inter predictor **221** or the intra predictor **222** from the input image signal (original block, original sample array) to generate a residual signal (residual block, residual sample array), and the generated residual signal is transmitted to the transformer **232**. In this case, as illustrated, a unit for subtracting the prediction signal (prediction block, prediction sample array) from an input image signal (original block, original sample array) in the encoder **200** may be referred to as a subtractor **231**. The predictor **220** may perform prediction on a processing target block (hereinafter, referred to as a current block) and generate a predicted block including prediction samples for the current block. The

predictor **220** may determine whether intra prediction or inter prediction is applied in units of a current block or CU. The predictor **220** may generate various kinds of information on prediction, such as prediction mode information, and transmit the generated information to the entropy encoder **240**, as is described below in the description of each prediction mode. The information on prediction may be encoded by the entropy encoder **240** and output in the form of a bitstream.

The intra predictor **222** may predict a current block with reference to samples within a current picture. The referenced samples may be located neighboring to the current block, or may also be located away from the current block according to the prediction mode. The prediction modes in the intra prediction may include a plurality of non-directional modes and a plurality of directional modes. The non-directional mode may include, for example, a DC mode or a planar mode. The directional mode may include, for example, 33 directional prediction modes or 65 directional prediction modes according to the fine degree of the prediction direction. However, this is illustrative and the directional prediction modes which are more or less than the above number may be used according to the setting. The intra predictor **222** may also determine the prediction mode applied to the current block using the prediction mode applied to the neighboring block.

The inter predictor **221** may induce a predicted block of the current block based on a reference block (reference sample array) specified by a motion vector on a reference picture. At this time, in order to decrease the amount of motion information transmitted in the inter prediction mode, the motion information may be predicted in units of a block, a sub-block, or a sample based on the correlation of the motion information between the neighboring block and the current block. The motion information may include a motion vector and a reference picture index. The motion information may further include inter prediction direction (L0 prediction, L1 prediction, Bi prediction, or the like) information. In the case of the inter prediction, the neighboring block may include a spatial neighboring block existing within the current picture and a temporal neighboring block existing in the reference picture. The reference picture including the reference block and the reference picture including the temporal neighboring block may also be the same as each other, and may also be different from each other. The temporal neighboring block may be called the name such as a collocated reference block, a collocated CU (colCU), or the like, and the reference picture including the temporal neighboring block may also be called a collocated picture (colPic). For example, the inter predictor **221** may configure a motion information candidate list based on the neighboring blocks, and generate information indicating what candidate is used to derive the motion vector and/or the reference picture index of the current block. The inter prediction may be performed based on various prediction modes, and for example, in the case of a skip mode and a merge mode, the inter predictor **221** may use the motion information of the neighboring block as the motion information of the current block. In the case of the skip mode, the residual signal may not be transmitted unlike the merge mode. A motion vector prediction (MVP) mode may indicate the motion vector of the current block by using the motion vector of the neighboring block as a motion vector predictor, and signaling a motion vector difference.

The predictor **220** may generate a prediction signal based on various prediction methods to be described below. For example, the predictor **220** may apply intra prediction or inter prediction for prediction of one block and may simultaneously apply intra prediction and inter prediction. This may be called combined inter and intra prediction (CIIP). In addition, the predictor may be based on an intra block copy (IBC) prediction mode or based on a palette mode for prediction of a block. The IBC prediction mode or the palette mode may be used for image/video coding of content such as games, for example, screen content coding (SCC). IBC basically performs prediction within the current picture, but may be performed similarly to inter prediction in that a reference block is derived within the current picture. That is, IBC may use at least one of the inter prediction techniques described in the present document. The palette mode may be viewed as an example of intra coding or intra prediction. When the palette mode is applied, a sample value in the picture may be signaled based on information on the palette table and the palette index.

The prediction signal generated by the predictor (including the inter predictor **221** and/or the intra predictor **222**) may be used to generate a reconstructed signal or may be used to generate a residual signal.

The transformer **232** may generate transform coefficients by applying a transform technique to the residual signal. For example, the transform technique may include at least one of a discrete cosine transform (DCT), a discrete sine transform (DST), a graph-based transform (GBT), or a conditionally non-linear transform (CNT). Here, GBT refers to transformation obtained from a graph when expressing relationship information between pixels in the graph. CNT refers to transformation obtained based on a prediction signal generated using all previously reconstructed pixels. Also, the transformation process may be applied to a block of pixels having the same size as a square or may be applied to a block of a variable size that is not a square.

The quantizer **233** quantizes the transform coefficients and transmits the same to the entropy encoder **240**, and the entropy encoder **240** encodes the quantized signal (information on the quantized transform coefficients) and outputs the encoded signal as a bitstream. Information on the quantized transform coefficients may be referred to as residual information. The quantizer **233** may rearrange the quantized transform coefficients in the block form into a one-dimensional vector form based on a coefficient scan order and may generate information on the transform coefficients based on the quantized transform coefficients in the one-dimensional vector form.

The entropy encoder **240** may perform various encoding methods such as, for example, exponential Golomb, context-adaptive variable length coding (CAVLC), and context-adaptive binary arithmetic coding (CABAC). The entropy encoder **240** may encode information necessary for video/image reconstruction (e.g., values of syntax elements, etc.) other than the quantized transform coefficients together or separately. Encoded information (e.g., encoded video/image information) may be transmitted or stored in units of a network abstraction layer (NAL) unit in the form of a bitstream. The video/image information may further include information on various parameter sets, such as an adaptation parameter set (APS), a picture parameter set (PPS), a sequence parameter set (SPS), or a video parameter set (VPS). Also, the video/image information may further include general constraint information. In the present document, information and/or syntax elements transmitted/signaled from the encoding apparatus to the decoding apparatus may be included in video/image information. The video/image information may be encoded through the encoding procedure described above and included in the bitstream.

The bitstream may be transmitted through a network or may be stored in a digital storage medium. Here, the network may include a broadcasting network and/or a communication network, and the digital storage medium may include various storage media such as USB, SD, CD, DVD, Blu-ray, HDD, and SSD. A transmitting unit (not shown) and/or a storing unit (not shown) for transmitting or storing a signal output from the entropy encoder **240** may be configured as internal/external elements of the encoding apparatus **200**, or the transmitting unit may be included in the entropy encoder **240**.

The quantized transform coefficients output from the quantizer **233** may be used to generate a prediction signal. For example, the residual signal (residual block or residual samples) may be reconstructed by applying dequantization and inverse transform to the quantized transform coefficients through the dequantizer **234** and the inverse transform unit **235**. The adder **250** may add the reconstructed residual signal to the prediction signal output from the inter predictor **221** or the intra predictor **222** to generate a reconstructed signal (reconstructed picture, reconstructed block, reconstructed sample array). When there is no residual for the processing target block, such as when the skip mode is applied, the predicted block may be used as a reconstructed block. The adder **250** may be referred to as a restoration unit or a restoration block generator. The generated reconstructed signal may be used for intra prediction of a next processing target block in the current picture, or may be used for inter prediction of the next picture after being filtered as described below.

Meanwhile, luma mapping with chroma scaling (LMCS) may be applied during a picture encoding and/or reconstruction process.

The filter **260** may improve subjective/objective image quality by applying filtering to the reconstructed signal. For example, the filter **260** may generate a modified reconstructed picture by applying various filtering methods to the reconstructed picture, and store the modified reconstructed picture in the memory **270**, specifically, in a DPB of the memory **270**. The various filtering methods may include, for example, deblocking filtering, a sample adaptive offset, an adaptive loop filter, a bilateral filter, and the like. The filter **260** may generate various kinds of information related to the filtering, and transfer the generated information to the entropy encoder **240** as described later in the description of each filtering method. The information related to the filtering may be encoded by the entropy encoder **240** and output in the form of a bitstream.

The modified reconstructed picture transmitted to the memory **270** may be used as a reference picture in the inter predictor **221**. When the inter prediction is applied through the encoding apparatus, prediction mismatch between the encoding apparatus **200** and the decoding apparatus may be avoided and encoding efficiency may be improved.

The DPB of the memory **270** may store the modified reconstructed picture for use as the reference picture in the inter predictor **221**. The memory **270** may store motion information of a block from which the motion information in the current picture is derived (or encoded) and/or motion information of blocks in the picture, having already been reconstructed. The stored motion information may be transferred to the inter predictor **221** to be utilized as motion information of the spatial neighboring block or motion information of the temporal neighboring block. The memory **270** may store reconstructed samples of reconstructed blocks in the current picture, and may transfer the reconstructed samples to the intra predictor **222**.

FIG. **3** is a diagram for schematically explaining the configuration of a video/image decoding apparatus to which the embodiments of the present document may be applied.

Referring to FIG. **3**, the decoding apparatus **300** may include and configured with an entropy decoder **310**, a residual processor **320**, a predictor **330**, an adder **340**, a filter **350**, and a memory **360**. The predictor **330** may include an inter predictor **331** and an intra predictor **332**. The residual processor **320** may include a dequantizer **321** and an inverse transformer **322**. The entropy decoder **310**, the residual processor **320**, the predictor **330**, the adder **340**, and the filter **350**, which have been described above, may be configured by one or more hardware components (e.g., decoder chipsets or processors) according to an embodiment. Further, the memory **360** may include a decoded picture buffer (DPB), and may be configured by a digital storage medium. The hardware component may further include the memory **360** as an internal/external component.

When the bitstream including the video/image information is input, the decoding apparatus **300** may reconstruct the image in response to a process in which the video/image information is processed in the encoding apparatus illustrated in FIG. **2**. For example, the decoding apparatus **300** may derive the units/blocks based on block split-related information acquired from the bitstream. The decoding apparatus **300** may perform decoding using the processing unit applied to the encoding apparatus. Therefore, the processing unit for the decoding may be, for example, a coding unit, and the coding unit may be split according to the quad-tree structure, the binary-tree structure, and/or the ternary-tree structure from the coding tree unit or the maximum coding unit. One or more transform units may be derived from the coding unit. In addition, the reconstructed image signal decoded and output through the decoding apparatus **300** may be reproduced through a reproducing apparatus.

The decoding apparatus **300** may receive a signal output from the encoding apparatus of FIG. **2** in the form of a bitstream, and the received signal may be decoded through the entropy decoder **310**. For example, the entropy decoder **310** may parse the bitstream to derive information (e.g., video/image information) necessary for image reconstruction (or picture reconstruction). The video/image information may further include information on various parameter sets such as an adaptation parameter set (APS), a picture parameter set (PPS), a sequence parameter set (SPS), or a video parameter set (VPS). In addition, the video/image information may further include general constraint information. The decoding apparatus may further decode picture based on the information on the parameter set and/or the general constraint information. Signaled/received information and/or syntax elements described later in the present document may be decoded may decode the decoding procedure and obtained from the bitstream. For example, the entropy decoder **310** decodes the information in the bitstream based on a coding method such as exponential Golomb coding, context-adaptive variable length coding (CAVLC), or context-adaptive arithmetic coding (CABAC), and output syntax elements required for image reconstruction and quantized values of transform coefficients for residual. More specifically, the CABAC entropy decoding method may receive a bin corresponding to each syntax element in the bitstream, determine a context model by using a decoding target syntax element information, decoding information of a decoding target block or information of a symbol/bin decoded in a previous stage, and perform an arithmetic decoding on the bin by predicting a probability of

occurrence of a bin according to the determined context model, and generate a symbol corresponding to the value of each syntax element. In this case, the CABAC entropy decoding method may update the context model by using the information of the decoded symbol/bin for a context model of a next symbol/bin after determining the context model. The information related to the prediction among the information decoded by the entropy decoder **310** may be provided to the predictor (inter predictor **332** and intra predictor **331**), and residual values on which the entropy decoding has been performed in the entropy decoder **310**, that is, the quantized transform coefficients and related parameter information, may be input to the residual processor **320**.

The residual processor **320** may derive a residual signal (residual block, residual samples, or residual sample array). Also, information on filtering among the information decoded by the entropy decoder **310** may be provided to the filter **350**. Meanwhile, a receiving unit (not shown) for receiving a signal output from the encoding apparatus may be further configured as an internal/external element of the decoding apparatus **300**, or the receiving unit may be a component of the entropy decoder **310**. Meanwhile, the decoding apparatus according to the present document may be called a video/image/picture decoding apparatus, and the decoding apparatus may be divided into an information decoder (video/image/picture information decoder) and a sample decoder (video/image/picture sample decoder). The information decoder may include the entropy decoder **310**, and the sample decoder may include at least one of the dequantizer **321**, the inverse transformer **322**, the adder **340**, the filter **350**, the memory **360**, an inter predictor **332**, and an intra predictor **331**.

The dequantizer **321** may dequantize the quantized transform coefficients to output the transform coefficients. The dequantizer **321** may rearrange the quantized transform coefficients in a two-dimensional block form. In this case, the rearrangement may be performed based on a coefficient scan order performed by the encoding apparatus. The dequantizer **321** may perform dequantization for the quantized transform coefficients using a quantization parameter (e.g., quantization step size information), and acquire the transform coefficients.

The inverse transformer **322** inversely transforms the transform coefficients to acquire the residual signal (residual block, residual sample array).

The predictor **330** may perform the prediction of the current block, and generate a predicted block including the prediction samples of the current block. The predictor may determine whether the intra prediction is applied or the inter prediction is applied to the current block based on the information on prediction output from the entropy decoder **310**, and determine a specific intra/inter prediction mode.

The predictor **330** may generate a prediction signal based on various prediction methods to be described later. For example, the predictor may apply intra prediction or inter prediction for prediction of one block, and may simultaneously apply intra prediction and inter prediction. This may be called combined inter and intra prediction (CIIP). In addition, the predictor may be based on an intra block copy (IBC) prediction mode or based on a palette mode for prediction of a block. The IBC prediction mode or the palette mode may be used for image/video coding of content such as games, for example, screen content coding (SCC). IBC may basically perform prediction within the current picture, but may be performed similarly to inter prediction in that a reference block is derived within the current picture. That is, IBC may use at least one of the inter prediction techniques described in the present document. The palette mode may be considered as an example of intra coding or intra prediction. When the palette mode is applied, information on the palette table and the palette index may be included in the video/image information and signaled.

The intra predictor **331** may predict the current block by referring to the samples in the current picture. The referred samples may be located in the neighborhood of the current block, or may be located apart from the current block according to the prediction mode. In intra prediction, prediction modes may include a plurality of non-directional modes and a plurality of directional modes. The intra predictor **331** may determine the prediction mode to be applied to the current block by using the prediction mode applied to the neighboring block.

The inter predictor **332** may derive a predicted block for the current block based on a reference block (reference sample array) specified by a motion vector on a reference picture. In this case, in order to reduce the amount of motion information being transmitted in the inter prediction mode, motion information may be predicted in the unit of blocks, subblocks, or samples based on correlation of motion information between the neighboring block and the current block. The motion information may include a motion vector and a reference picture index. The motion information may further include information on inter prediction direction (L0 prediction, L1 prediction, Bi prediction, and the like). In case of inter prediction, the neighboring block may include a spatial neighboring block existing in the current picture and a temporal neighboring block existing in the reference picture. For example, the inter predictor **332** may construct a motion information candidate list based on neighboring blocks, and derive a motion vector of the current block and/or a reference picture index based on the received candidate selection information. Inter prediction may be performed based on various prediction modes, and the information on the prediction may include information indicating a mode of inter prediction for the current block.

The adder **340** may generate a reconstructed signal (reconstructed picture, reconstructed block, or reconstructed sample array) by adding the obtained residual signal to the prediction signal (predicted block or predicted sample array) output from the predictor (including inter predictor **332** and/or intra predictor **331**). If there is no residual for the processing target block, such as a case that a skip mode is applied, the predicted block may be used as the reconstructed block.

The adder **340** may be called a reconstructor or a reconstructed block generator. The generated reconstructed signal may be used for the intra prediction of a next block to be processed in the current picture, and as described later, may also be output through filtering or may also be used for the inter prediction of a next picture.

Meanwhile, a luma mapping with chroma scaling (LMCS) may also be applied in the picture decoding process.

The filter **350** may improve subjective/objective image quality by applying filtering to the reconstructed signal. For example, the filter **350** may generate a modified reconstructed picture by applying various filtering methods to the reconstructed picture, and store the modified reconstructed picture in the memory **360**, specifically, in a DPB of the memory **360**. The various filtering methods may include, for example, deblocking filtering, a sample adaptive offset, an adaptive loop filter, a bilateral filter, and the like.

The (modified) reconstructed picture stored in the DPB of the memory **360** may be used as a reference picture in the

inter predictor **332**. The memory **360** may store the motion information of the block from which the motion information in the current picture is derived (or decoded) and/or the motion information of the blocks in the picture having already been reconstructed. The stored motion information may be transferred to the inter predictor **332** so as to be utilized as the motion information of the spatial neighboring block or the motion information of the temporal neighboring block. The memory **360** may store reconstructed samples of reconstructed blocks in the current picture, and transfer the reconstructed samples to the intra predictor **331**.

In the present document, the embodiments described in the filter **260**, the inter predictor **221**, and the intra predictor **222** of the encoding apparatus **200** may be applied equally or to correspond to the filter **350**, the inter predictor **332**, and the intra predictor **331**.

The video/image coding method according to the present document may be performed based on the following partitioning structure. Specifically, procedures of prediction, residual processing ((inverse) transform and (de)quantization), syntax element coding, and filtering to be described later may be performed based on CTU and CU (and/or TU and PU) derived based on the partitioning structure. A block partitioning procedure may be performed by the image partitioner **210** of the above-described encoding apparatus, and partitioning-related information may be (encoding) processed by the entropy encoder **240**, and may be transferred to the decoding apparatus in the form of a bitstream. The entropy decoder **310** of the decoding apparatus may derive the block partitioning structure of the current picture based on the partitioning-related information obtained from the bitstream, and based on this, may perform a series of procedures (e.g., prediction, residual processing, block/picture reconstruction, in-loop filtering, and the like) for image decoding. The CU size and the TU size may be equal to each other, or a plurality of TUs may be present within a CU region. Meanwhile, the CU size may generally represent a luma component (sample) coding block (CB) size. The TU size may generally represent a luma component (sample) transform block (TB) size. The chroma component (sample) CB or TB size may be derived based on the luma component (sample) CB or TB size in accordance with a component ratio according to a color format (chroma format, e.g., 4:4:4, 4:2:2, 4:2:0 and the like) of a picture/image. The TU size may be derived based on max TbSize. For example, if the CU size is larger than the maxTbSize, a plurality of TUs (TBs) of the max TbSize may be derived from the CU, and the transform/inverse transform may be performed in the unit of TU (TB). Further, for example, in case that intra prediction is applied, the intra prediction mode/type may be derived in the unit of CU (or CB), and neighboring reference sample derivation and prediction sample generation procedures may be performed in the unit of TU (or TB). In this case, one or a plurality of TUs (or TBs) may be present in one CU (or CB) region, and in this case, the plurality of TUs (or TBs) may share the same intra prediction mode/type.

Further, in the video/image coding according to the present document, an image processing unit may have a hierarchical structure. One picture may be partitioned into one or more tiles, bricks, slices, and/or tile groups. One slice may include one or more bricks. On brick may include one or more CTU rows within a tile. The slice may include an integer number of bricks of a picture. One tile group may include one or more tiles. One tile may include one or more CTUs. The CTU may be partitioned into one or more CUs. A tile represents a rectangular region of CTUs within a particular tile column and a particular tile row in a picture. A tile group may include an integer number of tiles according to a tile raster scan in the picture. A slice header may carry information/parameters that can be applied to the corresponding slice (blocks in the slice). In case that the encoding/decoding apparatus has a multi-core processor, encoding/decoding processes for the tiles, slices, bricks, and/or tile groups may be processed in parallel. In the present document, the slice or the tile group may be used exchangeably. That is, a tile group header may be called a slice header. Here, the slice may have one of slice types including intra (I) slice, predictive (P) slice, and bi-predictive (B) slice. In predicting blocks in I slice, inter prediction may not be used, and only intra prediction may be used. Of course, even in this case, signaling may be performed by coding the original sample value without prediction. With respect to blocks in P slice, intra prediction or inter prediction may be used, and in case of using the inter prediction, only uni-prediction can be used. Meanwhile, with respect to blocks in B slice, the intra prediction or inter prediction may be used, and in case of using the inter prediction, up to bi-prediction can be maximally used.

The encoder may determine the tile/tile group, brick, slice, and maximum and minimum coding unit sizes in consideration of the coding efficiency or parallel processing, or according to the characteristics (e.g., resolution) of a video image, and information for them or information capable of inducing them may be included in the bitstream.

The decoder may obtain information representing the tile/tile group, brick, and slice of the current picture, and whether the CTU in the tile has been partitioned into a plurality of coding units. By making such information be obtained (transmitted) only under a specific condition, the efficiency can be enhanced.

The slice header (slice header syntax) may include information/parameters that can be commonly applied to the slice. APS (APS syntax) or PPS (PPS syntax) may include information/parameters that can be commonly applied to one or more pictures. The SPS (SPS syntax) may include information/parameters that can be commonly applied to one or more sequences. The VPS (VPS syntax) may include information/parameters that can be commonly applied to multiple layers. The DPS (DPS syntax) may include information/parameters that can be commonly applied to overall video. The DPS may include information/parameters related to concatenation of a coded video sequence (CVS).

In the present document, an upper-level syntax may include at least one of the APS syntax, PPS syntax, SPS syntax, VPS syntax, DPS syntax, and slice header syntax.

Further, for example, information about partitioning and configuration of the tile/tile group/brick/slice may be configured by an encoding end through the upper-level syntax, and may be transferred to the decoding apparatus in the form of a bitstream.

In the present document, at least one of quantization/dequantization and/or transform/inverse transform may be omitted. When the quantization/dequantization is omitted, the quantized transform coefficient may be referred to as a transform coefficient. When the transform/inverse transform is omitted, the transform coefficient may be called a coefficient or a residual coefficient or may still be called the transform coefficient for uniformity of expression.

In the present document, the quantized transform coefficient and the transform coefficient may be referred to as a transform coefficient and a scaled transform coefficient, respectively. In this case, the residual information may include information on transform coefficient(s), and the information on the transform coefficient(s) may be signaled

through residual coding syntax. Transform coefficients may be derived based on the residual information (or information on the transform coefficient(s)), and scaled transform coefficients may be derived through inverse transform (scaling) on the transform coefficients. Residual samples may be derived based on inverse transform (transform) of the scaled transform coefficients. This may be applied/expressed in other parts of the present document as well.

As described above, the encoding apparatus may perform various encoding methods such as exponential Golomb, context-adaptive variable length coding (CAVLC), and context-adaptive binary arithmetic coding (CABAC). In addition, the decoding apparatus may decode information in a bitstream based on a coding method such as exponential Golomb coding, CAVLC or CABAC, and output a value of a syntax element required for image reconstruction and quantized values of transform coefficients related to residuals. For example, the coding methods described above may be performed as in the following contents.

FIG. **4** exemplarily shows context-adaptive binary arithmetic coding (CABAC) for encoding a syntax element.

An encoding process of CABAC may include a process of transforming an input signal into a binary value through binarization in case that the input signal is not the binary value, but is a syntax element. If the input signal is already the binary value (i.e., if the value of the input signal is the binary value), the corresponding input signal may be bypassed without being binarized. Here, each binary number 0 or 1 constituting the binary value may be called a bin. For example, if a binary string after binarization is 110, each of 1, 1, and 0 is called one bin. The bin(s) for one syntax element may indicate a value of the syntax element.

The binarized bins of the syntax element may be input to a regular encoding engine or a bypass encoding engine. The regular encoding engine may allocate a context model that reflects a probability value to the corresponding bin, and may code the corresponding bin based on the allocated context model. The regular encoding engine may update the context model for the corresponding bin after performing the coding for the respective bins. The bin being coded as described above may be called a context-coded bin.

Meanwhile, in case that the binarized bins of the syntax element are input to the bypass encoding engine, they may be coded as follows. For example, the bypass encoding engine of the encoding apparatus omits a procedure of estimating probability for the input bin and a procedure of updating a probability model applied to the bin after encoding. In case that the bypass encoding is applied, the encoding apparatus may encode the input bin by applying a uniform probability distribution instead of allocating the context model, and through this, the encoding speed can be improved. The bin being encoded as described above may be called a bypass bin.

Entropy decoding performs the same process as the entropy encoding as described above in reverse order. For example, in case that the syntax element is decoded based on the context model, the decoding apparatus may receive the bin corresponding to the syntax element through a bitstream. Further, the decoding apparatus may determine the context model using the syntax element, decoding information of a decoding target block or a neighboring block, or information of a symbol/bin decoded in a previous step, and may derive the value of the syntax element by performing arithmetic decoding of the bin through prediction of the probability of occurrence of the received bin according to the determined context model. Thereafter, the context model of the bin being next decoded may be updated with the determined context model.

Further, for example, in case that the syntax element is bypass-decoded, the decoding apparatus may receive the bin corresponding to the syntax element through the bitstream, and may decode the input bin by applying the uniform probability distribution. In this case, the procedure of deriving the context model of the syntax element and the procedure of updating the context model applied to the bin after decoding may be omitted.

The residual samples may be derived as quantized transform coefficients through transform and quantization processes. The quantized transform coefficients may be called transform coefficients. In this case, the transform coefficients in the block may be signaled in the form of residual information. The residual information may include a residual coding syntax. That is, the encoding apparatus may configure and encode the residual coding syntax based on the residual information to output the residual coding syntax in the form of a bitstream, and the decoding apparatus may derive the residual (quantized) transform coefficients by decoding the residual coding syntax obtained from the bitstream. As described below, the residual coding syntax may include syntax elements representing whether transform has been applied to the corresponding block, where is the location of the last effective transform coefficient in the block, whether the effective transform coefficient is present in the subblock, and what is the size/sign of the effective transform coefficient.

For example, the (quantized) transform coefficient may be encoded and/or decoded based on the syntax elements, such as last_sig_coeff_x_prefix, last_sig_coeff_y_prefix, last_sig_coeff_x_suffix, last_sig_coeff_y_suffix, coded_sub_block_flag, sig_coeff_flag, par_level_flag, abs_level_gtX_flag, abs_remainder, coeff_sign_flag, and dec_abs_level. This may be called residual (data) coding or (transform) coefficient coding. The syntax elements related to the encoding/decoding of the residual data may be represented as in Table 1 or Table 2 below.

TABLE 1

	Descriptor
residual_coding(x0, y0, log2TbWidth, log2TbHeight, cIdx) {	
if((tu_mts_idx[x0][y0] > 0 ||	
(cu_sbt_flag && log2TbWidth < 6 && log2TbHeight < 6))	
&& cIdx = = 0 && log2TbWidth > 4)	
log2ZoTbWidth = 4	
else	
log2ZoTbWidth = Min(log2TbWidth, 5)	
MaxCcbs = 2 * (1 << log2TbWidth) * (1<< log2TbHeight)	
if(tu_mts_idx[x0][y0] > 0 ||	
(cu_sbt_flag && log2TbWidth < 6 && log2TbHeight < 6))	
&& cIdx = = 0 && log2TbHeight > 4)	
log2ZoTbHeight = 4	

TABLE 1-continued

	Descriptor
else	
log2ZoTbHeight = Min(log2TbHeight, 5)	
if(log2TbWidth > 0)	
last_sig_coeff_x_prefix	ae(v)
if(log2TbHeight > 0)	
last_sig_coeff_y_prefix	ae(v)
if(last_sig_coeff_x_prefix > 3)	
last_sig_coeff_x_suffix	ae(v)
if(last_sig_coeff_y_prefix > 3)	
last_sig_coeff_y_suffix	ae(v)
log2TbWidth = log2ZoTbWidth	
log2TbHeight = log2ZoTbHeight	
remBinsPass1 = ((1 << (log2TbWidth + log2TbHeight)) * 7) >> 2	
log2SbW = (Min(log2TbWidth, log2TbHeight) < 2 ? 1 : 2)	
log2SbH = log2SbW	
if(log2TbWidth + log2TbHeight > 3) {	
if(log2TbWidth < 2) {	
log2SbW = log2TbWidth	
log2SbH = 4 − log2SbW	
} else if(log2TbHeight < 2) {	
log2SbH = log2TbHeight	
log2SbW = 4 − log2SbH	
}	
}	
numSbCoeff = 1 << (log2SbW + log2SbH)	
lastScanPos = numSbCoeff	
lastSubBlock = (1 << (log2TbWidth + log2TbHeight − (log2SbW + log2SbH))) − 1	
do {	
if(lastScanPos = = 0) {	
lastScanPos = numSbCoeff	
lastSubBlock− −	
}	
lastScanPos− −	
xS = DiagScanOrder[log2TbWidth − log2SbW][log2TbHeight − log2SbH] [lastSubBlock][0]	
yS = DiagScanOrder[log2TbWidth − log2SbW][log2TbHeight − log2SbH] [lastSubBlock][1]	
xC = (xS << log2SbW) + DiagScanOrder[log2SbW][log2SbH][lastScanPos][0]	
yC = (yS << log2SbH) + DiagScanOrder[log2SbW][log2SbH][lastScanPos][1]	
} while((xC != LastSignificantCoeffX) || (yC != LastSignificantCoeffY))	
if(lastSubBlock = = 0 && log2TbWidth >= 2 && log2TbHeight >= 2 &&	
!transform_skip_flag[x0][y0] && lastScanPos > 0)	
LfnstDcOnly = 0	
if((lastSubBlock > 0 && log2TbWidth >= 2 && log2Weight >= 2) ||	
(lastScanPos > 7 && (log2TbWidth = = 2 || log2TbWidth = = 3) &&	
log2TbWidth = = log2TbHeight))	
LfnstZeroOutSigCoeffFlag = 0	
QState = 0	
for(i = lastSubBlock; i >= 0; i− −) {	
startQStateSb = QState	
xS = DiagScanOrder[log2TbWidth − log2SbW][log2TbHeight − log2SbH] [i][0]	
yS = DiagScanOrder[log2TbWidth − log2SbW][log2TbHeight − log2SbH] [i][1]	
inferSbDcSigCoeffFlag = 0	
if((i < lastSubBlock) && (i > 0)) {	
coded_sub_block_flag[xS][yS]	ae(v)
inferSbDcSigCoeffFlag = 1	
}	
firstSigScanPosSb = numSbCoeff	
lastSigScanPosSb = −1	
firstPosMode0 = (i = = lastSubBlock ? lastScanPos : numSbCoeff − 1)	
firstPosMode1 = −1	
for(n = firstPosMode0; n >= 0 && remBinsPass1 >= 4; n− −) {	
xC = (xS << log2SbW) + DiagScanOrder[log2SbW][log2SbH][n][0]	
yC = (yS << log2SbH) + DiagScanOrder[log2SbW][log2SbH][n][1]	
if(coded_sub_block_flag[xS][yS] && (n < 0 || !inferSbDcSigCoeffFlag) &&	
(xC != LastSignificantCoeffX || yC != Last SignficantCoeffY)) {	
sig_coeff_flag[xC][yC]	ae(v)
remBinsPass1− −	
if(sig_coeff_flag[xC][yC])	
inferSbDcSigCoeffFlag = 0	
}	

TABLE 1-continued

	Descriptor
if(sig_coeff_flag[xC][yC]) {	
abs_level_gtx_flag[n][0]	ae(v)
remBinsPass1– –	
if(abs_level_gtx_flag[n][0]) {	
par_level_flag[n]	ae(v)
remBinsPass1– –	
abs_level_gtx_flag[n][1]	ae(v)
remBinsPass1– –	
}	
if(lastSigScanPosSb = = –1)	
lastSigScanPosSb = n	
firstSigScanPosSb = n	
}	
AbsLevelPass1[xC][yC] = sig_coeff_flag[xC][yC] + par_level_flag[n] +	
abs_level_gtx_flag[n][0] + 2 * abs_level_gtx_flag[n][1]	
if(dep_quant_enabled_flag)	
QState = QStateTransTable[QState][AbsLevelPass1[xC][yC] & 1]	
if(remBinsPass1 < 4)	
firstPosMode1 = n – 1	
}	
for(n = numSbCoeff – 1; n >= firstPosMode1; n– –) {	
xC = (xS << log2SbW) + DiagScanOrder[log2SbW][log2SbH][n][0]	
yC = (yS << log2SbH) + DiagScanOrder[log2SbW][log2SbH][n][1]	
if(abs_level_gtx_flag[n][1])	
abs_remainder[n]	ae(v)
AbsLevel[xC][yC] = AbsLevelPass1[xC][yC] +2 * abs_remainder[n]	
}	
for(n = firstPosMode1; n >= 0 : n– –) {	
xC = (xS << log2SbW) + DiagScanOrder[log2SbW][log2SbH][n][0]	
yC = (yS << log2SbH) + DiagScanOrder[log2SbW][log2SbH][n][1]	
dec_abs_level[n]	ae(v)
if(AbsLevel[xC][yC] > 0)	
firstSigScanPosSb = n	
if(dep_quant_enabled_flag)	
QState = QStateTransTable[QState][AbsLevel[xC][yC] & 1]	
}	
if(dep_quant_enabled_flag || !sign_data_hiding_enabled_flag)	
signHidden = 0	
else	
signHidden = (lastSigScanPosSb – firstSigScanPosSb > 3 ? 1 : 0)	
for(n = numSbCoeff – 1; n >= 0; n– –) {	
xC = (xS << log2SbW) + DiagScanOrder[log2SbW][log2SbH][n][0]	
yC = (yS << log2SbH) + DiagScanOrder[log2SbW][log2SbH][n][1]	
if((AbsLevel[xC][yC] > 0) &&	
(!signHidden || (n != firstSigScanPosSb)))	
coeff_sign_flag[n]	ae(v)
}	
if(dep_quant_enabled_flag) {	
QState = startQStateSb	
for(n = numSbCoeff – 1; n >= 0; n– –) {	
xC = (xS << log2SbW) + DiagScanOrder[log2SbW][log2SbH][n][0]	
yC = (yS << log2SbH) + DiagScanOrder[log2SbW][log2SbH][n][1]	
if(AbsLevel[xC][yC] > 0)	
TransCoeffLevel[x0][y0][cIdx][xC][yC] =	
(2 * AbsLevel[xC][yC] – (QState > 1 ? 1 : 0)) *	
(1 – 2 * coeff_sign_flag[n])	
QState = QStateTransTable[QState][par_level_flag[n]]	
} else {	
sumAbsLevel = 0	
for(n = numSbCoeff – 1; n >= 0; n– –) {	
xC = (xS << log2SbW) + DiagScanOrder[log2SbW][log2SbH][n][0]	
yC = (yS << log2SbH) + DiagScanOrder[log2SbW][log2SbH][n][1]	
if(AbsLevel[xC][yC] > 0) {	
TransCoeffLevel[x0][y0][cIdx][xC][yC] =	
AbsLevel[xC][yC] * (1 – 2 * coeff_sign_flag[n])	
if(signHidden) {	
sumAbsLevel += AbsLevel[xC][yC]	
if((n = = firstSigScanPosSb) && (sumAbsLevel % 2) = = 1))	
TransCoeffLevel[x0][y0][cIdx][xC][yC] =	
–TransCoeffLevel[x0][y0][cIdx][xC][yC]	
}	
}	
}	
}	
}	
}	

TABLE 2

```
                                                                    Descriptor
residual_cading( x0, y0, log2TbWidth, log2TbHeight, cIdx ) {
  if( sps_mts_enabled_flag && cu_sbt_flag && cIdx = = 0 &&
      log2TbWidth = = 5 && log2TbHeight < 6 )
    log2ZoTbWidth = 4
  else
    log2ZoTbWidth = Min( log2TbWidth, 5 )
  if( sps_mts_enabled_flag && cu_sbt_flag && cIdx = = 0 &&
      log2TbWidth < 6 && log2TbHeight = = 5 )
    log2ZoTbHeight = 4
  else
    log2ZoTbHeight = Min( log2TbHeight, 5 )
  if( log2TbWidth > 0 )
    last_sig_coeff_x_prefix                                         ae(v)
  if( log2TbHeight > 0 )
    last_sig_coeff_y_prefix                                         ae(v)
  if( last_sig_coeff_x_prefix > 3 )
    last_sig_coeff_x_suffix                                         ae(v)
  if( last_sig_coeff_y_prefix > 3 )
    last_sig_coeff_y_suffix                                         ae(v)
  log2TbWidth = log2ZoTbWidth
  log2TbHeight = log2ZoTbHeight
  remBinsPass1 = ( ( 1 << ( log2TbWidth + log2TbHeight ) ) * 7 ) >> 2
  log2SbW = ( Min( log2TbWidth, log2TbHeight ) < 2 ? 1 : 2 )
  log2SbH = log2SbW
  if( log2TbbWidth + log2TbHeight > 3 )
    if( log2TbWidth < 2 ) {
      log2SbW = log2TbWidth
      log2SbH = 4 − log2SbW
    } else if( log2TbHeight < 2 ) {
      log2SbH = log2TbHeight
      log2SbW = 4 − log2SbH
    }
  numSbCoeff = 1 << ( log2SbW + log2SbH )
  lastScanPos = numSbCoeff
  lastSubBlock = ( 1 << ( log2TbWidth + log2TbHeight − ( log2SbW + log2SbH ) ) ) − 1
  do {
    if( lastScanPos = = 0 ) {
      lastScanPos = numSbCoeff
      lastSubBlock− −
    }
    lastScanPos− −
    xS = DiagScanOrder[ log2TbWidth − log2SbW ][ log2TbHeight − log2SbH ]
                  [ lastSubBlock ][ 0 ]
    yS = DiagScanOrder[ log2TbWidth − log2SbW ][ log2TbHeight − log2SbH ]
                  [ lastSubBlock ][ 1 ]
    xC = ( xS << log2SbW ) + DiagScanOrder[ log2SbW ][ log2SbH ][ lastScanPos ][ 0 ]
    yC = ( yS << log2SbH ) + DiagScanOrder[ log2SbW ][ log2SbH ][ lastScanPos ][ 1 ]
  } while( ( xC != LastSignificantCoeffX ) || ( yC != LastSignificantCoeffY ) )
  if( lastSubBlock = = 0 && log2TbWidth >= 2 && log2TbHeight >= 2
&&
      !transform_skip_flag[ x0 ][ y0 ][ cIdx ] && lastScanPos > 0 )
    LfnstDcOnly = 0
  if( ( lastSubBlock > 0 && log2TbWidth >= 2 && log2TbHeight >= 2 ) ||
      ( lastScanPos > 7 && ( log2TbWidth = = 2 || log2TbWidth = = 3 )
&&
      log2TbWidth = = log2TbHeight ) )
    LfnstZeroOutSigCoeffFlag = 0
  if( ( lastSubBlock > 0 || lastScanPos > 0 ) && cIdx = = 0 )
    MtsDcOnly = 0
  QState = 0
  for( i = lastSubBlock; i >= 0; i− − ) {
    startQStateSb = QState
    xS = DiagScanOrder[ log2TbWidth − log2SbW ][ log2TbHeight − log2SbH ]
                  [ i ][ 0 ]
    yS = DiagScanOrder[ log2TbWidth − log2SbW ][ log2TbHeight − log2SbH ]
                  [ i ][ 1 ]
    inferSbDcSigCoeffFlag = 0
    if( i < lastSubBlock && i > 0 ) {
      sb_coded_flag[ xS ][ yS ]                                     ae(v)
      inferSbDcSigCoeffFlag = 1
    }
    if( sb_coded_flag[ xS ][ yS ] && ( xS > 3 || yS > 3 ) && cIdx = = 0 )
      MtsZeroOutSigCoeffFlag = 0
    firstSigScanPosSb = numSbCoeff
    lastSigScanPosSb = −1
    firstPosMode0 = ( i = = lastSubBlock ? lastScanPos : numSbCoeff − 1 )
    firstPosMode1 = firstPosMode0
```

TABLE 2-continued

	Descriptor
for(n = firstPosMode0; n >= 0 && remBinsPass1 >= 4; n– –) {	
xC = (xS << log2SbW) + diagScanOrder[log2SbW][log2SbH][n][0]	
yC = (yS << log2SbH) + diagScanOrder[log2SbW][log2SbH][n][1]	
if(sb_coed_flag[xS][yS] && (n > 0 || !inferSbDcSigCoeffFlag) &&	
(xC != LastSignificantCoeffX || yC != Last SignificantCoeffY)) {	
sig_coeff_flag[xC][yC]	ae(v)
remBinsPass1– –	
if(sig_coeff_flag[xC][yC]	
inferSbDcSigCoeffFlag = 0	
}	
if(sig_coeff_flag[xC][yC]) {	
abs_level_gtx_flag[n][0]	ae(v)
remBinsPass1– –	
if(abs_level_gtx_flag[n][0]) {	
par_level_flag[n]	ae(v)
remBinsPass1– –	
abs_level_gtx_flag[n][1]	ae(v)
remBinsPass1– –	
}	
if(lastSigScanPosSb = = –1)	
lastSigScanPosSb = n	
firstSigScanPosSb = n	
}	
AbsLevelPass1[xC][yC] = sig_coeff_flag[xC][yC] + par_level_flag[n] +	
abs_level_gtx_flag[n][0] + 2 * abs_level_gtx_flag[n][1]	
if(sh_dep_quant_used_flag)	
QState = QStateTransTable[QState][AbsLevelPass1[xC][yC] & 1]	
firstPosMode1 = n – 1	
}	
for(n = firstPosMode0; n > firstPosMode1; n– –) {	
xC = (xS << log2SbW) + DiagScanOrder[log2SbW][log2SbH][n][0]	
yC = (yS << log2SbH) + DiagScanOrder[log2SbW][log2SbH][n][1]	
if(abs_level_gtx_flag[n][1])	
abs_remainder[n]	ae(v)
AbsLevel[xC][yC] = AbsLevelPass1[xC][yC] +2 * abs_remainder[n]	
}	
for(n = firstPosMode1; n >= 0; n– –) {	
xC = (xS << log2SbW) + DiagScanOrder[log2SbW][log2SbH][n][0]	
yC = (yS << log2SbH) + DiagScanOrder[log2SbW][log2SbH][n][1]	
if(sb_coded_flag[xS][yS])	
dec_abs_level[n]	ae(v)
if(AbsLevel[xC][yC] > 0) {	
if(lastSigScanPosSb = = –1)	
lastSigScanPosSb = n	
firstSigScanPosSb = n	
}	
if(sh_dep_quant_used_flag)	
QState = QStateTransTable[QState][AbsLevel[xC][yC] & 1]	
}	
signHiddenFlag = sh_sign_data_hiding_used_flag &&	
(lastSigScanPosSb – firstSigScanPosSb > 3 ? 1 : 0)	
for(n = numSbCoeff – 1; n >= 0; n– –) {	
xC = (xS << log2SbW) + DiagScanOrder[log2SbW][log2SbH][n][0]	
yC = (yS << log2SbH) + DiagScanOrder[log2SbW][log2SbH][n][1]	
if((AbsLevel[xC][yC] > 0) &&	
(!signHiddenFlag || (in != firstSigScanPosSb)))	
coeff_sign_flag[n]	ae(v)
}	
if(sh_dep_quant_used_flag) {	
QState = startQStateSb	
for(n = numSbCoeff – 1; n >= 0; n– –) {	
xC = (xS << log2SbW) + DiagScanOrder[log2SbW][log2SbH][n][0]	
yC = (yS << log2SbH) + DiagScanOrder[log2SbW][log2SbH][n][1]	
if(AbsLevel[xC][yC] > 0)	
TransCoeffLevel[x0][y0][cIdx][xC][yC] =	
(2 * AbsLevel[xC][yC] – (QState > 1 ? 1 : 0)) *	
(1 – 2 * coeff_sign_flag[n])	
QState = QStateTransTable[QState][AbsLevel[xC][yC] & 1]	
} else {	
sumAbsLevel = 0	
for(n = numSbCoeff – 1; n >= 0; n– –) {	
xC = (xS << log2SbW) + DiagScanOrder[log2SbW][log2SbH][n][0]	
yC = (yS << log2SbH) + DiagScanOrder[log2SbW][log2SbH][n][1]	
if(AbsLevel[xC][yC] > 0) {	
TransCoeffLevel[x0][y0][cIdx][xC][yC] =	
AbsLevel[xC][yC] * (1 – 2 * coeff_sign_flag[n])	

TABLE 2-continued

	Descriptor

if(signHiddenFlag) {
sumAbsLevel += AbsLevel[xC][yC]
if((n = = firstSigScanPosSb) && (sumAbsLevel % 2) = = 1))
TransCoeffLevel[x0][y0][cIdx][xC][yC] =
−TransCoeffLevel[x0][y0][cIdx][xC][yC]
}
}
}
}
}
}	

In Table 1 and Table 2, transform_skip_flag represents whether the transform for the associated block is omitted. The transform_skip_flag may be a syntax element of the transform skip flag. The associated block may be a coding block (CB) or a transform block (TB). With respect to the transform (and quantization) and residual coding procedure, the CB and the TB may be interchangeably used. For example, residual samples may be derived with respect to the CB, and (quantized) transform coefficients may be derived through the transform and quantization for the residual samples. Through the residual coding procedure, information (e.g., syntax elements) efficiently representing the location, size, sign, and the like of the (quantized) transform coefficients may be generated and signaled. The quantized transform coefficients may be simply called transform coefficients. In general, if the CB is not larger than the maximum TB, the size of the CB may be equal to the size of the TB, and in this case, the target block being transformed (and quantized) and residual-coded may be called the CB or the TB. Meanwhile, if the CB is larger than the maximum TB, the target block being transformed (and quantized) and residual-coded may be called the TB. Hereinafter, although it is explained that the syntax elements related to the residual coding are signaled in the unit of a transform block (TB), this is merely exemplary, and the TB may be interchangeably used with the CB as described above.

The syntax for the residual coding according to the transform skip flag may be the same as that in Table 3 or Table 4.

TABLE 3

	Descriptor

residual_ts_coding(x0, y0, log2TbWidth, log2TbHeight, cIdx) {
log2SbSize = (Min(log2TbWidth, log2TbHeight) < 2 ? 1 : 2)
numSbCoeff = 1 << (log2SbSize << 1)
lastSubBlock = (1 << (log2TbWidth + log2TbHeight − 2 * log2SbSize)) − 1
inferSbCbf = 1
MaxCcbs = 2 * (1 << log2TbWidth) * (1 << log2TbHeight)
for(i =0; i <= lastSubBlock; i++) {
xS = DiagScanOrder[log2TbWidth − log2SbSize][log2TbHeight − log2SbSize][i][0]
yS = DiagScanOrder[log2TbWidth − log2SbSize][log2TbHeight − log2SbSize][i][1]
if((i != lastSubBlock || !inferSbCbf) {
coded_sub_block_flag[xS][yS]
	ae(v)

}
if(coded_sub_block_flag[xS][yS] && i < lastSubBlock)
inferSbCbf = 0
/* First scan pass */
inferSbSigCoeffFlag = 1
for(n = 0; n <= numSbCoeff − 1; n++) {
xC = (xS << log2SbSize) + DiagScanOrder[log2SbSize][log2SbSize][n][0]
yC = (yS << log2SbSize) + DiagScanOrder[log2SbSize][log2SbSize][n][1]
if(coded_sub_block_flag[xS][yS] &&
(n != numSbCoeff − 1 || !inferSbSigCoeffFlag)) {
sig_coeff_flag[xC][yC]
	ae(v)

MaxCcbs− −
if(sig_coeff_flag[xC][yC])
inferSbSigCoeffFlag = 0
}
CoeffSignLevel[xC][yC] = 0
if(sig_coeff_flag[xC][yC] {
coeff_sign_flag[n]
	ae(v)

MaxCcbs− − CoeffSignLevel[xC][yC] = (coeff_sign_flag[n] > 0 ? −1 : 1) abs_level_gtx_flag[n][0]	ae(v)
MaxCcbs− − if(abs_level_gtx_flag[n][0]) { par_level_flag[n]	ae(v)
MaxCcbs− − } }	

TABLE 3-continued

```
                                                                                    Descriptor
        AbsLevelPassX[ xC ][ yC ] =
            sig_coeff_flag[ xC ][ yC ] + par_level_flag[ n ] + abs_level_gtx_flag[ n ][ 0 ]
      }
    /* Greater than X scan pass (numGtXFlags=5) */
      for( n = 0; n <= numSbCoeff - 1; n++ ) {
        xC = (xS << log2SbSize) + DiagScanOrder[ log2SbSize ][ log2SbSize ][ n ][ 0 ]
        yC = (yS << log2SbSize) + DiagScanOrder[ log2SbSize ][ log2SbSize ][ n ][ 1 ]
        for( j = 1; j < 5; j++ ) {
          if( abs_level_gtx_flag[ n ][ j - 1 ] )
            abs_level_gtx_flag[ n ][ j ]                                            ae(v)
          MaxCcbs- -
          AbsLevelPassX[ xC ][ yC ] + = 2 * abs_level_gtx_flag[ n ][ j ]
        }
      }
    /* remainder scan pass */
      for( n = 0; n <= numSbCoeff - 1; n++ ) {
        xC = ( xS << log2SbSize ) + DiagScanOrder[ log2SbSize ][ log2SbSize ][ n][ 0 ]
        yC = ( yS << log2SbSize ) + DiagScanOrder[ log2SbSize ][ log2SbSize ][ n][ 1 ]
        if( abs_level_gtx_flag[ n ][ 4 ] )
          abs_remainder[ n ]                                                        ae(v)
        if( intra_bdpcm_flag = = 0 ) {
          absRightCoeff = abs( TransCoeffLevel[ x0 ][ y0 ][ cIdx ][ xC - 1 ][ yC ] )
          absBelowCoeff = abs( TransCoeffLevel[ x0 ][ y0 ][ cIdx ][ xC ][ yC - 1 ] )
          predCoeff = Max( absRightCoeff, absBelowCoeff )
          if( AbsLevelPassX[ xC ][ yC ] + abs_remainder[ n ] = = 1 && predCoeff >
0 )
            TransCoeffLevel[ x0 ][ y0 ][ cIdx ][ xC ][ yC ] =
              ( 1 - 2 * coeff_sign_flag[ n ] ) * predCoeff
          else if( AbsLevelPassX[ xC ][ yC ] + abs_remainder[ n ] <= predCoeff )
            TransCoeffLevel[ x0 ][ y0 ][ cIdx ][ xC ][ yC ] = ( 1 - 2 * coeff_sign_flag[ n ] ) *
              ( AbsLevelPassX[ xC ][ yC ] + abs_remainder[ n ] - 1 )
          else
            TransCoeffLevel[ x0 ][ y0 ][ cIdx ][ xC ][ yC ] = ( 1 - 2 * coeff_sign_flag[ n ] ) *
              ( AbsLevelPassX[ xC ][ yC ] + abs_remainder[ n ] )
        } else
          TransCoeffLevel[ x0 ][ y0 ][ cIdx ][ xC ][ yC ] = ( 1 - 2 * coeff_sign_flag[ n ] ) *
            ( AbsLevelPassX[ xC ][ yC ] + abs_remainder[ n ] )
      }
    }
}
```

TABLE 4

```
                                                                                    Descriptor
residual_ts_coding( x0, y0, log2TbWidth, log2TbHeight, cIdx ) {
  log2SbW = ( Min( log2TbWidth, log2TbHeight ) < 2 ? 1 : 2 )
  log2SbH = log2SbW
  if( log2TbWidth + log2TbHeight > 3 )
    if( log2TbWidth < 2 ) {
      log2SbW = log2TbWidth
      log2SbH = 4 - log2SbW
    } else if( log2TbHeight < 2 ) {
      log2SbH = log2TbHeight
      log2SbW = 4 - log2SbH
    }
  numSbCoeff = 1 << ( log2SbW + log2SbH )
  lastSubBlock = ( 1 << ( log2TbWidth + log2TbHeight - ( log2SbW + log2SbH ) ) ) - 1
  inferSbCbf = 1
  RemCcbs = ( ( 1 << ( log2TbWidth + log2TbHeight ) ) * 7 ) >> 2
  for( i =0; i <= lastSubBlock; i++ ) {
    xS = DiagScanOrder[ log2TbWidth - log2SbW ][ log2TbHeight - log2SbH ][ i ][ 0 ]
    yS = DiagScanOrder[ log2TbWidth - log2SbW ][ log2TbHeight - log2SbH ][ i ][ 1 ]
    if( i != lastSubBlock || !inferSbCbf )
      sb_coded_flag[ xS ][ yS ]                                                     ae(v)
    if( sb_coded_flag[ xS ][ yS ] && i < lastSubBlock )
      inferSbCbf = 0
  /* First scan pass */
    inferSbSigCoeffFlag = 1
    lastScanPosPass1 = -1
    for( n 0; n <= numSbCoeff - 1 && RemCcbs >= 4; n++ ) {
      xC = ( xS << log2SbW ) + DiagScanOrder[ log2SbW ][ log2SbH ][ n ][ 0 ]
      yC = ( yS << log2SbH ) + DiagScanOrder[ log2SbW ][ log2SbH ][ n ][ 1 ]
      lastScanPosPass1 = n
```

TABLE 4-continued

	Descriptor
if(sb_coded_flag[xS][yS] &&	
(n != numSbCoeff − 1 || !inferSbSigCoeffFlag)) {	
sig_coeff_flag[xC][yC]	ae(v)
RemCcbs− −	
if(sig_coeff_flag[xC][yC])	
inferSbSigCoeffFlag = 0	
}	
CoeffSignLevel[xC][yC] = 0	
if(sig_coeff_flag[xC][yC]) {	
coeff_sign_flag[n]	ae(v)
RemCcbs− −	
CoeffSignLevel[xC][yC] = (coeff_sign_flag[n] > 0 ? −1 : 1)	
abd_level_gtx_flag[n][0]	ae(v)
RemCcbs− −	
if(abs_level_gtx_flag[n][0]) {	
par_level_flag[n]	ae(v)
RemCcbs− −	
}	
}	
AbsLevelPass[xC][yC] =	
sig coeff flag[xC][yC] + par level flag[n] + abs level gtx flag[n][0]	
}	
/* Greater than X scan pass (numGtXFlag=5) */	
lastScanPosPass2 = −1	
for(n = 0; n <= numSbCoeff − 1 && RemCcbs >= 4; n++) {	
xC = (xS << log2SbW) + DiagScanOrder[log2SbW][log2SbH][n][0]	
yC = (yS << log2SbH) + DiagScanOrder[log2SbW][log2SbH][n][1]	
AbsLevelPass2[xC][yC] = AbsLevelPass1[xC][yC]	
for(j = 1; j < 5; j++) {	
if(abs_level_gtx_flag[n][j − 1]) {	
abd_level_gtx_flag[n][j]	ae(v)
RemCcbs− −	
}	
AbsLevelPass2[xC][yC] += 2 * abs_level_gtx_flag[n][j]	
}	
lastScanPosPass2 = n	
}	
/* remainder scan pass */	
for(n = 0; n <= numSbCoeff − 1; n++) {	
xC = (xS << log2SbW) + DiagScanOrder[log2SbW][log2SbH][n][0]	
yC = (yS << log2SbH) + DiagScanOrder[log2SbW][log2SbH][n][1]	
if((n <= lastScanPosPass2 && AbsLevelPass2[xC][yC] >= 10) ||	
(n > lastScanPosPass2 && n <= lastScanPass1 &&	
AbsLevelPass1[xC][yC] >= 2) ||	
(n > lastScanPass1 && sb coded flag[xS][yS]))	
abs_remainder[n]	ae(v)
if(n <= lastScanPosPass2)	
AbsLevel[xC][yC] = AbsLevelPass2[xC][yC] + 2 * abs_remainder[n]	
else if (n <= lastScanPosPass1)	
AbsLevel[xC][yC] = AbsLevelPass1[xC][yC] + 2 * abs_remainder[n]	
else { /* bypass */	
AbsLevel[xC][yC] = abs_remainder[n]	
if(abs_remainder[n])	
coeff_sign_lag[n]	ae(v)
}	
if(BdpcmFlag[x0][y0][cIdx] = = 0 && n <= lastScanPosPass1) {	
absLeftCoeff = xC > 0 ? AbsLevel[xC − 1][yC]) : 0	
absAboveCoeff = yC > 0 ? AbsLevel[xC][yC − 1]) : 0	
predCoeff = Max(absLeftCoeff, absAboveCoeff)	
if(AbsLevel[xC][yC] = = 1 && predCoeff > 0)	
AbsLevel[xC][yC] = predCoeff	
else if(AbBLevel[xC][yC] > 0 && AbsLevel[xC][yC] <= predCoeff)	
AbsLevel[xC][yC]− −	
}	
TransCoeffLevel[x0][y0][cIdx][xC][yC] = (1 − 2 * coeff_sign_flag[n]) *	
AbsLevel[xC][yC]	
}	
}	
}	

According to the present embodiment, the residual coding may be branched according to the value of the transform skip flag transform_skip_flag. That is, different syntax elements may be used for the residual coding based on the value of the transform skip flag (based on whether to skip the transform). The residual coding being used in case that the transform skip is not applied (i.e., in case that the transform is applied) may be called regular residual coding (RRC), and the residual coding in case that the transform skip is not applied (i.e., in case that the transform is not applied) may be called transform skip residual coding (TSRC). Further, the regular residual coding may also be called general

residual coding. Further, the regular residual coding may be called a regular residual coding syntax structure, and the transform skip residual coding may be called a transform skip residual coding syntax structure. Table 1 and Table 2 may represent a residual coding syntax element in case that the value of transform_skip_flag is 0, that is, in case that the transform is applied, and Table 3 and Table 4 may represent a residual coding syntax element in case that the value of transform_skip_flag is 1, that is, in case that the transform is not applied.

Specifically, as an example, the transform skip flag indicating whether the transform of the transform block is skipped may be parsed, and whether the transform skip flag is 1 may be determined. If the value of the transform skip flag is 0, syntax elements last_sig_coeff_x_prefix, last_sig_coeff_y_prefix, last_sig_coeff_x_suffix, last_sig_coeff_y_suffix, sb_coded_flag, sig_coeff_flag, abs_level_gtx_flag, par_level_flag, abs_remainder, dec_abs_level, and/or coeff_sign_flag for residual coefficients of the transform block as illustrated in Table 1 or Table 2 may be parsed, and the residual coefficients may be derived based on the syntax elements. In this case, the syntax elements may be sequentially parsed, and the parsing order may be changed. Further, the abs_level_gtx_flag may represent the abs_level_gt1_flag and/or the abs_level_gt3_flag. For example, the abs_level_gtx_flag[n][0] may be an example of the first transform coefficient level flag abs_level_gt1_flag, and the abs_level_gtx_flag[n][1] may be an example of the second transform coefficient level flag abs_level_gt3_flag.

In an embodiment, the encoding apparatus may encode (x, y) position information of the last non-zero transform coefficient in a transform block based on the syntax elements last_sig_coeff_x_prefix, last_sig_coeff y_prefix, last_sig_coeff_x_suffix, and last_sig_coeff_y_suffix. More specifically, the last_sig_coeff_x_prefix represents a prefix of a column position of a last significant coefficient in a scanning order within the transform block, the last_sig_coeff_y_prefix represents a prefix of a row position of the last significant coefficient in the scanning order within the transform block, the last_sig_coeff_x_suffix represents a suffix of a column position of the last significant coefficient in the scanning order within the transform block, and the last_sig_coeff_y_suffix represents a suffix of a row position of the last significant coefficient in the scanning order within the transform block. Here, the significant coefficient may represent a non-zero coefficient. In addition, the scanning order may be a right diagonal scanning order. Alternatively, the scanning order may be a horizontal scanning order or a vertical scanning order. The scanning order may be determined based on whether intra/inter prediction is applied to a target block (a CB or a CB including a TB) and/or a specific intra/inter prediction mode.

Thereafter, the encoding apparatus may divide the transform block into 4×4 sub-blocks, and then indicate whether there is a non-zero coefficient in the current sub-block using a 1-bit syntax element coded_sub_block_flag for each 4×4 sub-block.

If a value of coded_sub_block_flag is 0, there is no more information to be transmitted, and thus, the encoding apparatus may terminate the encoding process on the current sub-block. Conversely, if the value of coded_sub_block_flag is 1, the encoding apparatus may continuously perform the encoding process on sig_coeff_flag. Since the sub-block including the last non-zero coefficient does not require encoding for the coded_sub_block_flag and the sub-block including the DC information of the transform block has a high probability of including the non-zero coefficient, coded_sub_block_flag may not be coded and a value thereof may be assumed as 1.

If the value of coded_sub_block_flag is 1 and thus it is determined that a non-zero coefficient exists in the current sub-block, the encoding apparatus may encode sig_coeff_flag having a binary value according to a reverse scanning order. The encoding apparatus may encode the 1-bit syntax element sig_coeff_flag for each transform coefficient according to the scanning order. If the value of the transform coefficient at the current scan position is not 0, the value of sig_coeff_flag may be 1. Here, in the case of a subblock including the last non-zero coefficient, sig_coeff_flag does not need to be encoded for the last non-zero coefficient, so the coding process for the sub-block may be omitted. Level information coding may be performed only when sig_coeff_flag is 1, and four syntax elements may be used in the level information encoding process. More specifically, each sig_coeff_flag[xC][yC] may indicate whether a level (value) of a corresponding transform coefficient at each transform coefficient position (xC, yC) in the current TB is non-zero. In an embodiment, the sig_coeff_flag may correspond to an example of a syntax element of a significant coefficient flag indicating whether a quantized transform coefficient is a non-zero significant coefficient.

The remaining level value after encoding of the sig_coeff_flag may be derived as in the following equation. That is, the syntax element remAbsLevel representing the level value to be encoded may be derived as in the following equation.

$$remAbsLevel[n] = |\ coeff[n]\ | - 1 \quad \text{[Equation 1]}$$

Here, coeff[n] means a real transform coefficient value.

Further, the abs_level_gtx_flag[n][0] may represent whether the remAbsLevel[n] at a corresponding scanning position n is larger than 1. For example, if the value of the abs_level_gtx_flag[n][0] is 0, an absolute value of the transform coefficient at the corresponding position may be 1. Further, if the value of the abs_level_gtx_flag[n][0] is 1, the remAbsLevel[n] representing the level value to be encoded hereinafter may be updated as in the equation below.

$$remAbsLevel[n] = remAbsLevel[n] - 1 \quad \text{[Equation 2]}$$

Further, the value of the least significant coefficient (LSB) of the remAbsLevel[n] described in Equation 2 may be encoded through the par_level_flag as in Equation 3 below.

$$par\_level\_flag[n] = remAbsLevel[n] \,\&1 \quad \text{[Equation 3]}$$

Here, par_level_flag[n] may represent a parity of the transform coefficient level (value) at the scanning position n.

After the par_leve_flag[n] is encoded, the transform coefficient level value remAbsLevel[n] to be encoded may be updated as in the following equation.

$$remAbsLevel[n] = remAbsLevel[n] >> 1 \quad \text{[Equation 4]}$$

abs_level_gtx_flag[n][1] may represent whether the remAbsLevel at the corresponding scanning position n is larger than 3. Only in case that the abs_level_gtx_flag[n][1] is 1, encoding of the abs_remainder[n] can be performed. The relationship between the real transform coefficient value coeff and each syntax element may be as in the following equation.

$$|coeff[n]| = sig\_coeff\_flag[n] + abs\_level\_gtX\_flag[n][0] + par\_level\_flag[n] + 2*(abs\_level\_gtX\_flag[n][1] + abs\_remainder[n]) \quad \text{[Equation 5]}$$

Further, the following Table 5 represents examples related to the above-described Equation 5.

TABLE 5

|coeff[n]|	sig_coeff_flag[n]	abs_level_gtX_flag[n][0]	par_level_flag[n]	abs_level_gtX_flag[n][1]	abs_remainder[n]
0	0				
1	1	0			
2	1	1	0	0	
3	1	1	1	0	
4	1	1	0	1	0
5	1	1	1	1	0
6	1	1	0	1	1
7	1	1	1	1	1
8	1	1	0	1	2
9	1	1	1	1	2
10	1	1	0	1	3
11	1	1	1	1	3
...	...	...	...		

Here, |coeff[n]| represents the transform coefficient level (value), and may be indicated as AbsLevel for the transform coefficient. Further, the sign of each coefficient may be encoded using coeff_sign_flag that is a one-bit symbol.

Further, as another example, if the value of the transform skip flag is 1, syntax elements sb_coded_flag, sig_coeff_flag, coeff_sign_flag, abs_level_gtx_flag, par_level_flag, and/or abs_remainder for the residual coefficient of the transform block as shown in Table 3 or Table 4 may be parsed, and the residual coefficient may be derived based on the syntax elements. In this case, the syntax elements may be sequentially parsed, or the parsing order may be changed. Further, the abs_level_gtx_flag may represent abs_level_gt1_flag, abs_level_gt3_flag, abs_level_gt5_flag, abs_level_gt7_flag, and/or abs_level_gt9_flag. For example, abs_level_gtx_flag[n][j] may be a flag representing whether an absolute value or level (value) of the transform coefficient at the scanning position n is larger than (j<<1)+1. In some cases, the (j<<1)+1 may be replaced by a predetermined threshold value, such as a first threshold value or a second threshold value.

Meanwhile, the CABAC provides high performance, but has the disadvantage that the throughput performance is not good. This is caused by the regular encoding engine of the CABAC, and since the regular encoding (encoding through the regular encoding engine of the CABAC) uses the probability state and range updated through encoding of the previous bin, it may show high data dependency, and it may take a lot of time to read the probability interval and to determine the current state. The throughput problem of the CABAC may be solved by limiting the number of context-coded bins. For example, as in the above-described Table 5, the sum of bins used to express the sig_coeff_flag[n], abs_level_gtx_flag[n][0], par_level_flag[n], and abs_level_gtx_flag[n][1] may be limited to 1.75 per pixel in the transform block depending on the size of the transform block. In this case, if all of the limited number of context-coded bins are used to encode the context element, the encoding apparatus may perform the bypass coding by binarizing the remaining coefficients through a binarization method to be described later without using the context coding. In other words, if the number of coded context-coded bins becomes TU width*TU height*1.75 in TU, the sig_coeff_flag[n], abs_level_gtx_flag[n][0], par_level_flag[n], and abs_level_gtx_flag[n][1], which are coded as the context-coded bins, may not be coded any more, and the value of |coeff[n]| may be directly coded into dec_abs_level[n] as in the following Table 6.

TABLE 6

|coeff[n]|	dec_abs_level[n]
0	0
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
10	10
11	11
...	...

In this case, the sign of each coefficient may be coded using coeff_sign_flag[n] that is a one-bit symbol.

FIG. **5** exemplarily illustrates transform coefficients in a 4×4 block.

The 4×4 block of FIG. **5** represents an example of quantized coefficients. The block illustrated in FIG. **5** may be a 4×4 transform block, or may be a 4×4 subblock of an 8×8, 16×16, 32×32, or 64×64 transform block. The 4×4 block of FIG. **5** may represent a luma block or a chroma block. However, this is merely exemplary, and in the present embodiment, the transform of a large block size (maximum 64×64 size) is possible, and this may be useful for a high-resolution video (e.g., 1080p and 4K sequence). The high-frequency transform coefficients may be zeroed with respect to the transform block of which the size (width or height or both width and height) is 64, and only low-frequency coefficients may be maintained. For example, in case of the M×N transform block having the block width of

M and the block height of N, only 32 left columns of the transform coefficients may be maintained when M is 64. Further, only 32 top rows of the transform coefficients may be maintained when N is 64.

In case that the transform skip mode is used for a block having a large size, the whole block may be used without being zeroed even with respect to any values. Since the maximum transform size configurable in SPS is supported, the encoding apparatus may adaptively select the transform size having a length of 16, 32, or 64 at maximum in accordance with necessity of a specific implementation. Specifically, binarization of the coefficient position coding that is not the last 0 may be coded based on the reduced TU size, and the selection of the context model for the coefficient position coding that is not the last 0 may be determined by the original TU size.

In FIG. **5**, as an example, the encoding results for the coefficients being scanned reverse diagonally are illustrated. In FIGS. **5**, n (0 to 15) designates the scan position of the coefficients according to the reverse diagonal scan. If n is 15, it represents the coefficient at the bottom right corner, which is firstly scanned in the 4×4 block, and if n is 0, it represent the coefficient at the top left corner, which is lastly scanned.

Meanwhile, as described above, if the input signal is not a binary value, but is the syntax element, the encoding apparatus may transform the input signal into a binary value by binarizing the value of the input signal. Further, the decoding apparatus may derive the binarized value (i.e., binarized bin) of the syntax element by decoding the syntax element, and may derive the value of the syntax element by reversely binarizing the binarized value. The binarization process may be performed as a truncated rice (TR) binarization process, a k-th order Exp-Golomb (EGk) binarization process, a limited k-th order Exp-Golomb (limited EGk) or fixed-length (FL) binarization process to be described later. Further, the reverse binarization process may be performed based on the TR binarization process, the EGk binarization process, the limited EGk binarization process, or the FL binarization process, and may represent a process of deriving the value of the syntax element.

For example, the TR binarization process may be performed as follows.

An input of the TR binarization process may be a request for TR binarization and cMax and cRiceParam for the syntax element. Further, an output of the TR binarization process may be the TR binarization for a value symbolVal corresponding to a bin string.

As an example, if a suffix bin string for the syntax element is present, the TR bin string for the syntax element may be a concatenation of a prefix bin string and a suffix bin string, and if the suffix bin string is not present, the TR bin string for the syntax element may be the prefix bin string. For example, the prefix bin string may be derived as follows.

The prefix value of the symbolVal may be derived as in the following equation.

$$prefixVal = symbolVal >> cRiceParam \quad \text{[Equation 6]}$$

Here, prefix Val may represent a prefix value of symbolVal. The prefix of the TR bin string (i.e., prefix bin string) may be derived as follows.

For example, if the prefix Val is smaller than cMax>>cRiceParam, the prefix bin string may be a bit string having a length of prefix Val+1, which is indexed by binIdx. That is, if the prefix Val is smaller than cMax>>cRiceParam, the prefix bin string may be a bit string having the number of bits prefix Val+1 indicated by the binIdx. The bin for the binIdx that is smaller the prefix Val may be 1. Further, the bin for the binIdx equal to the prefix Val may be 0.

For example, a bin string derived as unary binarization for the prefix Val may be as in Table 7 below.

TABLE 7

prefixVal	Bin string					
0	0					
1	1	0				
2	1	1	0			
3	1	1	1	0		
4	1	1	1	1	0	
5	1	1	1	1	1	0
. . .						
binIdx	0	1	2	3	4	5

Meanwhile, if the prefix Val is not smaller than cMax>>cRiceParam, the prefix bin string may be a bit string of which the length is the cMax>>cRiceParam and of which all bins 5 are 1.

Further, the suffix bin string of the TR bin string may be present in case that the cMax is larger than the symbolVal and the cRiceParam is larger than 0. For example, the prefix bin string may be derived as follows.

The suffix value of the symbolVal for the syntax element may be derived as in the following equation.

$$suffixVal = symbolVal - ((prefixVal) << cRiceParam) \quad \text{[Equation 7]}$$

Here, the suffix Val may represent a suffix value of the symbolVal.

The suffix (i.e., suffix bin string) of the TR bin string may be derived based on an FL binarization process for the suffix Val of which the cMax value is 1<<cRiceParam)−1.

For the input parameter cRiceParam=0, the TR binarization is exactly a truncated unary binarization and it is always invoked with a cMax value equal to the largest possible value of the syntax element being decoded.

Meanwhile, the EGk binarization process may be performed as follows.

The input of the EGk binarization process may be a request for the EGk binarization. Further, the output of the EGk binarization process may be the EGk binarization for the value symbolVal corresponding to the bin string.

The bit string of the EGk binarization process for the symbolVal may be derived as follows.

TABLE 8

```
absV = Abs( symbolVal )
stopLoop = 0
do
  if( absV >= ( 1 << k ) ) {
    put( 1 )
    absV = absV − ( 1 << k )
    k++
  } else {
    put( 0 )
    while( k— )
      put( ( absV >> k ) & 1 )
    stopLoop = 1
  }
while( !stopLoop )
```

Referring to Table 8, a binary value X may be added at the end of a bin string through each call of put(X). Here, X may be 0 or 1.

Further, a limited EGk binarization process may be performed as follows.

The input of the limited EGk binarization process may be a request for the limited EGk binarization and the rice parameter riceParam. Further, the output of the limited EGk binarization process may be the limited EGk binarization for the value symbolVal related to the corresponding bin string.

The bin string of the limited EGk binarization process for the symbolVal may be derived as follows.

TABLE 9

```
codeValue = symbolVal >> riceParam
preExtLen = 0
while( ( preExtLen < maxPreExtLen ) && ( codeValue > ( ( 2 << preExtLen ) − 2 ) ) ) {
  preExtLen++
  put( 1 )
}
if( preExtLen == maxPreExtLen )
  escapeLength = log2TransformRange
else {
  escapeLength = preExtLen + riceParam
  put( 0 )
}
symbolVal = symbolVal − ( ( ( 1 << preExtLen ) − 1 ) << riceParam )
while( ( escapeLength−− ) > 0 )
  put( ( symbolVal >> escapeLength ) & 1 )
```

Referring to Table 9, a binary value X may be added at the end of the bin string through each call of put(X). Here, X may be 0 or 1.

Variables log2TransformRange and maxPrefixExtensionLength may be derived as follows.

$$log2TransformRange = 15 \quad \text{[Equation 8]}$$

$$maxPrefixExtensionLength = 26 - log2TransformRange$$

Further, the FL binarization process may be performed as follows.

The input of the FL binarization process may be a request for the FL binarization and cMax for the syntax element. Further, the output of the FL binarization process may be the FL binarization for the value symbolVal corresponding to the bin string.

The FL binarization may be configured using the bin string having the number of bits of a fixed length of the symbol value symbolVal. Here, the fixed length may be derived as in the following equation.

$$fixedLength = Ceil(Log2(cMax + 1)) \quad \text{[Equation 9]}$$

That is, the bin string for the symbol value symbolVal may be derived through the FL binarization, and the bin length (i.e., the number of bits) of the bin string may be the fixed length.

Indexing of bins for the FL binarization may be a method for using a value increasing in order from the most significant bit to the least significant bit. For example, the bin index related to the most significant bit may be binIdx=0.

Meanwhile, a binarization process for a syntax element abs_remainder[n] among residual information may be performed as follows.

The input of the binarization process for the abs_remainder[n] may be a request for binarization of the syntax element abs_remainder[n], color component cIdx, luma position (x0, y0) representing the top left sample of the current luma transform block based on the top left luma sample of a picture, current coefficient scan position (xC, yC), binary logarithm of the transform block width longTbWidth, and binary logarithm of the transform block height log2TbHeight. The output of the binarization process for the abs_remainder may be binarization of the abs_remainder (i.e., binarized bin string of the abs_remainder). Through the binarization process, available bin strings for the abs_remainder may be derived.

The rice parameter cRiceParam for the abs_remainder[n] may be derived through a rice parameter deriving process being performed through inputting of the color component cIdx and the luma position (x0, y0), the current coefficient scan position (xC, yC), log2TbWidth that is the binary logarithm of the width of the transform block, and log2TbHeight that is the binary logarithm of the height of the transform block. The rice parameter deriving process will be described in detail later.

The cMax for the abs_remainder[n] being currently coded may be derived based on the rice parameter cRiceParam. For example, the cMax may be derived as in the following equation.

$$cMax = 6 << cRiceParam \quad \text{[Equation 10]}$$

Meanwhile, in case that the suffix bin string is present, the binarization for the syntax element abs_remainder[n], that is, the bin string for the abs_remainder[n] may be the concatenation of the prefix bin string and the suffix bin string. In case that the suffix bin string is not present, the bin string for the abs_remainder[n] may be the prefix bin string.

For example, the prefix bin string for the abs_remainder [n] may be derived as follows.

The prefix value prefix Val of the abs_remainder[n] may be derived as in the following equation.

$$prefixVal = Min(cMax, abs\_remainder[n]) \quad \text{[Equation 11]}$$

The prefix bin string of the abs_remainder[n] may be derived through the TR binarization process for the prefix Val using the cMax and the cRiceParam as an input.

If the prefix bin string is equal to a bit string of which all bits are 1 and the bit length is 6, the suffix bin string of the abs_remainder[n] may be present, and may be derived as follows.

The suffix value suffix Val of the abs_remainder[n] may be derived as in the following equation.

$$suffixVal = \text{abs\_remainder}[\ n\ ] - cMax \quad \text{[Equation 12]}$$

The suffix bin string of the abs_remainder[n] may be derived through the limited EGk binarization process for the binarization of the suffix Val that uses cRiceParam+1 and cRiceParam as an input.

The rice parameter for the abs_remainder[n] may be derived by the following process.

The input of the rice parameter deriving process may be a base level baseLevel, a color component index cIdx, a luma position (x0, y0), a current coefficient scan position (xC, yC), a binary logarithm log2TbWidth of the width of the transform block and a binary logarithm log2TbHeight of the height of the transform block. The luma position (x0, y0) may represent the top left sample of the current luma transform block based on the top left luma sample of the picture. The output of the rice parameter deriving process may be the rice parameter cRiceParam.

For example, the variable locSumAbs may be derived by a pseudo code disclosed in the following table based on the arrangement AbsLevel[x][y] for the given component index cIdx and the transform block having the top left luma position (x0, y0).

TABLE 10

```
- If transform_skip_flag[ x0 ][ y0 ] is equal to 1, trafoSkip is set equal to 1 and the following
  applies:
  locSumAbs = 0
  if( xC > 0 )
    locSumAbs += AbsLevel[ xC − 1 ][ yC ]
  if( yC > 0 )
    locSumAbs += AbsLevel[ xC ][ yC − 1 ]
  locSumAbs = Clip3( 0, 31, locSumAbs )
- Otherwise (transform_skip_flag[ x0 ][ y0 ] is equal to 0), trafoSkip is set equal to 0 and the
  following applies:
  locSumAbs = 0
  if ( xC < (1 << log2TbWidth) − 1 ) {
    locSumAbs += AbsLevel[ xC + 1 ][ yC ]
    if( xC < (1 << log2TbWidth) − 2 )
      locSumAbs += AbsLevel[ xC + 2 ][ yC ]
    if( yC < (1 << log2TbHeight) − 1 )
      locSumAbs += AbsLevel[ xC + 1 ][ yC + 1 ]
  }
  if( yC < (1 << log2TbHeight) − 1 ) {
    locSumAbs += AbsLevel[ xC ][ yC + 1 ]
    if( yC < (1 << log2TbHeight) − 2 )
      locSumAbs += AbsLevel [ xC ][ yC + 2 ]
  }
  locSumAbs = Clip3( 0, 31, locSumAbs − baseLevel * 5 )
```

In case that baseLevel is 0 in Table 10, the variable s may be configured as Max (0,QState−1), and the rice parameter cRiceParam and the variable ZeroPos[n] may be derived as in the following Table 11 based on the variables locSumAbs, trafoSkip, and s. If the baseLevel is larger than 0, the rice parameter cRiceParam may be derived as in Table 11 based on the variables locSumAbs and trafoSkip.

TABLE 11

		locSumAbs															
trafoSkip	s	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	cRiceParam	0	0	0	0	0	0	0	1	1	1	1	1	1	1	2	2
1	cRiceParam	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1
	0 ZeroPos[n]	0	0	0	0	0	1	2	2	2	2	2	2	4	4	4	4
	1 ZeroPos[n]	1	1	1	1	2	3	4	4	4	6	6	6	8	8	8	8
	2 ZeroPos[n]	1	1	2	2	2	3	4	4	4	6	6	6	8	8	8	8

		locSumAbs															
		16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
0	cRiceParam	2	2	2	2	2	2	2	2	2	2	2	2	3	3	3	3
1	cRiceParam	1	1	1	1	1	1	1	1	1	2	2	2	2	2	2	2
	0 ZeroPos[n]	4	4	4	4	4	4	4	8	8	8	8	8	16	16	16	16
	1 ZeraPos[n]	4	4	12	12	12	12	12	12	12	12	16	16	16	16	16	16
	2 ZeroPos[n]	8	8	12	12	12	12	12	12	12	16	16	16	16	16	16	16

Meanwhile, the binarization process for the syntax element dec_abs_level among the residual information may be performed as follows.

The input of the binarization process for the dec_abs_level may be a request for the binarization of the syntax element dec_abs_level[n], a color component cIdx, a luma position (x0, y0), a current coefficient scan position (xC, yC), a binary logarithm log2TbWidth of the width of the transform block, and a binary logarithm log2TbHeight of the height of the transform block. The luma position (x0, y0) may represent the top left sample of the current luma transform block based on the top left luma sample of the picture.

The output of the binarization process for the dec_abs_level may be binarization of the dec_abs_level (i.e., binarized bin string of the dec_abs_level). Through the binarization process, available bin strings for the dec_abs_level may be derived.

The rice parameter cRiceParam for the dec_abs_level[n] may be derived through the rice parameter deriving process that is performed through inputting of the color component cIdx and the luma position (x0, y0), the current coefficient scan position (xC, yC), the log2TbWidth that is the binary logarithm of the width of the transform block, and the log2TbHeight that is the binary logarithm of the height of the transform block. The rice parameter deriving process will be described in detail later.

Further, for example, the cMax for the dec_abs_level[n] may be derived based on the rice parameter cRiceParam. The cMax may be derived as in Equation 10.

Meanwhile, in case that the suffix bin string is present, the binarization for the dec_abs_level[n], that is, the bin string for the dec_abs_level[n], may be the concatenation of the prefix bin string and the suffix bin string. Further, in case that the suffix bin string is not present, the bin string for the dec_abs_level[n] may be the prefix bin string.

For example, the prefix bin string may be derived as follows.

The prefix value prefix Val of the dec_abs_level[n] may be derived as in the following equation.

$$prefixVal = \text{Min}(\ cMax, \text{dec\_abs\_level}[\ n\ ]\ ) \quad \text{[Equation 13]}$$

The prefix bin string of the dec_abs_level[n] may be derived through the TR binarization process for the prefix Val that uses the cMax and the cRiceParam as an input.

If the prefix bin string is equal to the bit string of which all bits are 1 and the bit length is 6, the suffix bin string of the dec_abs_level[n] may be present, and this may be derived as follows.

The suffix value suffix Val of the dec_abs_level[n] may be derived as in the following equation.

$$suffixVal = \text{dec\_abs\_level}[\ n\ ] - cMax \quad \text{[Equation 14]}$$

The suffix bin string of the dec_abs_level[n] may be derived through the limited EGk binarization process for the binarization of the suffix Val of which the Exp-Golomb degree K is configured as cRiceParam+1.

The rice parameter for the dec_abs_level[n] may be derived by the pseudo code of Table 10.

Meanwhile, the regular residual coding (RRC) and the transform skip residual coding (TSRC) as described above may have the following difference.

For example, the rice parameter cRiceParam of the syntax element abs_remainder[ ] in the regular residual coding may be derived as described above, but the rice parameter cRiceParam of the syntax element abs_remainder[ ] in the transform skip residual coding may be derived as 1. That is, for example, in case that the transform skip is applied to the current block (e.g., current TB), the rice parameter cRiceParam for the abs_remainder[ ] of the transform skip residual coding for the current block may be derived as 1.

Further, referring to Table 1 to Table 4, although abs_level_gtx_flag[n][**0**] and/or abs_level_gtx_flag[n][1] may be signaled in the regular residual coding, abs_level_gtx_flag[n][0], abs_level_gtx_flag[n][1], abs_level_gtx_flag[n][2], abs_level_gtx_flag[n][3], and abs_level_gtx_flag[n][4] may be signaled in the transform skip residual coding. Here, the abs_level_gtx_flag[n][0] may be represented as abs_level_gt1_flag or the first coefficient level flag, the abs_level_gtx_flag[n][1] may be represented as abs_level_gt3_flag or the second coefficient level flag, the abs_level_gtx_flag[n][2] may be represented as abs_level_gt5_flag or the third coefficient level flag, the abs_level_gtx_flag[n][3] may be represented as abs_level_gt7_flag or the fourth coefficient level flag, and the abs_level_gtx_flag[n][4] may be represented as abs_level_gt9_flag or the fifth coefficient level flag. Specifically, the first coefficient level flag may be a flag representing whether the coefficient level is larger than the first threshold value (e.g., 1), the second coefficient level flag may be a flag representing whether the coefficient level is larger than the second threshold value (e.g., 3), the third coefficient level flag may be a flag representing whether the coefficient level is larger than the third threshold value (e.g., 5), the fourth coefficient level flag may be a flag representing whether the coefficient level is larger than the fourth threshold value (e.g., 7), and the fifth coefficient level flag may be a flag representing whether the coefficient level is larger than the fifth threshold value (e.g., 9).

As described above, in comparison to the regular residual coding, the transform skip residual coding may further include abs_level_gtx_flag[n][2], abs_level_gtx_flag[n][3], and abs_level_gtx_flag[n][4] in addition to abs_level_gtx_flag[n][0] and abs_level_gtx_flag[n][1].

Further, for example, in the regular residual coding, the syntax element coeff_sign_flag may be bypass-coded, whereas in the transform skip residual coding, the syntax element coeff_sign_flag may be bypass-coded or context-coded.

The following description has been prepared to explain specific examples of the present document. Hereinafter, since the name of a specific device or the name of specific signal/information is exemplarily presented, the technical feature of the present specification is not limited to the specific names used in the following explanation.

Hereinafter, disclosed is a method for efficiently deriving a rice parameter for binarization of information representing a level value (or an absolute value) of a transform coefficient in level coding (e.g., a syntax element dec_abs_level representing the level value of the transform coefficient and a syntax element abs_remainder representing a residual level value of the transform coefficient).

A rice parameter is a variable that is used to binarize the level value of the transform coefficient, and if residual data coding (regular residual coding) for a transform block is applied to the current block, a rice parameter lookup table as

in the following Table 12 is used, whereas if residual data coding (transform skip residual coding) for a transform skip block is applied to the current block, a rice parameter lookup table as in the following Table 13 is used. Here, the rice parameter lookup table may be called a table about the rice parameter or a table that is used to determine the rice parameter. Alternatively, the rice parameter lookup table may be called a table about rice parameter candidates.

TABLE 12

	locSumAbs															
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
cRiceParam	0	0	0	0	0	0	0	1	1	1	1	1	1	1	2	2
	locSumAbs															
	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
cRiceParam	2	2	2	2	2	2	2	2	2	2	2	2	3	3	3	3

TABLE 13

	locSumAbs															
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
cRiceParam	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1
	locSumAbs															
	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
cRiceParam	1	1	1	1	1	1	1	1	1	2	2	2	2	2	2	2

In Table 12 and Table 13, locSumAbs is a value being derived based on the sum of level value(s) of neighboring transform coefficient(s) of the current transform coefficient, and for example, it may be derived by a pseudo code of Table 10.

In binarizing the level value, the encoding apparatus and the decoding apparatus generate a codeword that is advantageous to the binarization of a small level value by allocating a short codeword to the small level value as the value of the rice parameter cRiceParam becomes smaller, and generate a codeword that is advantageous to the binarization of a large level value by allocating a short codeword to the large level value as the value of the rice parameter becomes larger.

However, in accordance with the characteristic of an image and/or the existence/nonexistence or the type of the syntax being coded prior to the level value of the transform coefficient, level values generated on average by the level coding or level values being frequently generated are different from each other. Further, in case of lossless (or near-lossless) coding or in a high bit-rate environment being coded with a low quantization parameter, level values having different characteristics may be mixed. In this case, in order to derive the rice parameter, it may be more efficient to use a plurality of rice parameter lookup tables rather than to use one rice parameter lookup table.

Accordingly, according to an embodiment, two or more rice parameter lookup tables may be used for a case that the residual data coding (regular residual coding) for the transform block is applied to the current block and for a case that the residual data coding (transform skip residual coding) for the transform skip block is applied to the current block, respectively. Further, two or more rice parameter lookup tables may be used regardless of whether the regular residual coding or the transform skip residual coding is applied to the current block.

In this case, as an example, the encoding apparatus may derive the rice parameter by selecting at least one of a plurality of rice parameter lookup tables based on the configuration of the syntax element (the existence/nonexistence or the type of the syntax element) that is coded prior to the syntax element (e.g., dec_abs_level or abs_remainder) representing the level value of the transform coefficient. Alternatively, the encoding apparatus may derive the rice parameter by selecting at least one of the plurality of rice parameter lookup tables based on whether the number of context coded bins encoded/decoded in the residual data coding exceeds the maximum number of available context coded bins configured by a context coded bin constraint algorithm.

Here, the plurality of rice parameter lookup tables may have different rice parameter minimum values or different rice parameter maximum values, and may have different update locations of the rice parameter value. Further, the syntax element being coded prior to the syntax element representing the level value of the transform coefficient may include at least one of sig_coeff_flag, abs_level_gtx_flag, par_level_flag, or coeff_sign_flag. Further, the maximum number of available context coded bins configured by the context coded bin constraint algorithm may correspond to remBinsPass1 of Table 1 or Table 2 in case of the residual coding for the transform block, and may correspond to MaxCcbs of Table 3 in case of the residual coding for the transform skip block.

As an example, in case that two rice parameter lookup tables are present, the first rice parameter lookup table "A" may have m_A as the minimum value of the rice parameter, and may have n_A as the maximum value of the rice parameter. The second rice parameter lookup table "B" may have m_B as the minimum value of the rice parameter, and may have n_B as the maximum value of the rice parameter. In order to improve the performance of the level coding, m_B may be

configured as a larger value than m_A, and n_B may be configured as a larger value than n_A. For example, the first rice parameter lookup table may be configured as in the following Table 14, and the second rice parameter lookup table may be configured as in the following Table 15.

TABLE 14

	locSumAbs															
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
cRiceParam	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1
	locSumAbs															
	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
cRiceParam	1	1	1	1	1	1	1	1	1	2	2	2	2	2	2	2

TABLE 15

	locSumAbs															
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
cRiceParam	1	1	1	1	1	1	1	2	2	2	2	2	2	2	2	2
	locSumAbs															
	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
cRiceParam	2	2	3	3	3	3	3	3	3	3	3	3	3	3	3	3

In Table 14, the minimum value and the maximum value of the rice parameter are 0 and 2, respectively, and in Table 15, the minimum value and the maximum value of the rice parameter are 1 and 3, respectively. That is, the rice parameter lookup table of Table 15 has the minimum value and the maximum value which are larger than those of the rice parameter lookup table of Table 14. Further, the rice parameter value in Table 14 is updated when the locSumAbs value is 12 and 24, and the rice parameter value in Table 15 is updated when the locSumAbs value is 7 and 18. That is, the rice parameter lookup table of Table 14 and the rice parameter lookup table of Table 15 have different update locations of the rice parameter value.

Accordingly, as compared with the rice parameter lookup table of Table 14, the rice parameter lookup table of Table 15 may take advantage in case that a relatively large level value appears frequently or in case that an average of level values or coefficients of a lower block being coded is large.

The rice parameter lookup tables of Table 14 and Table 15 are merely examples of various rice parameter lookup tables that can be used according to the present embodiment, and in case of applying the present embodiment, the rice parameter lookup tables are not limited thereto, and tables having different rice parameter minimum values, maximum values, and update locations may be used.

Meanwhile, the encoding apparatus may signal the syntax element (or flag) for transmitting information on the rice parameter lookup table being used for the current transform coefficient among the plurality of rice parameter lookup tables. The syntax element may be signaled in the unit of a coefficient group (CG), or may be signaled in the unit of a transform block or a transform skip (coding) block. As an example, in case of using two rice parameter lookup tables, if the syntax element value is 0, the first rice parameter lookup table may be represented, and if the syntax element value is 1, the second rice parameter lookup table may be represented. The syntax element may be binarized through one of various methods, such as a fixed length binarization and a truncated unary binarization.

If the syntax element (or flag) representing the information on the rice parameter lookup table is obtained from the bitstream, the decoding apparatus may select the rice parameter lookup table represented by the syntax element among the plurality of rice parameter lookup tables, and may derive the rice parameter for the current transform coefficient based on this.

Further, the decoding apparatus may infer or derive the rice parameter lookup table being used for the level coding of the current transform coefficient (or coefficient group, transform block, or coding block) among the plurality of rice parameter lookup tables by using already given information, such as the configuration of the syntax element (the existence/nonexistence or the type of the syntax element) being coded prior to the syntax element (e.g., dec_abs_level or abs_remainder) representing the level value of the current transform coefficient, whether the number of encoded/decoded context coded bins in the residual data coding exceeds the maximum number of available context coded bins configured in the context coded bin constraint algorithm, whether lossless or near-lossless coding is performed, or quantization coefficient information. In this case, the syntax element or the flag representing the information on the rice parameter lookup table used for the current transform coefficient may not be signaled.

As an example, whether the number of encoded/decoded context coded bins in the residual data coding exceeds the maximum number (remBinsPass1 or MaxCcbs) of available context coded bins configured in the context coded bin constraint algorithm may be used to select the rice parameter lookup table. In this case, if the number of encoded/decoded context coded bins in the residual data coding exceeds the maximum number of available context coded bins, a sim-

plified syntax configuration that is different from the existing syntax configuration being used in the residual coding may be used.

For example, in the residual data coding for the transform block, (i) if the number of encoded/decoded context coded bins does not exceed the remBinsPass1, sig_coeff_flag, abs_level_gtx_flag[0], par_level_flag, and abs_level_gtx_flag[1] are coded prior to the abs_remainder representing the residual level value of the transform coefficient. However, (ii) if the number of encoded/decoded context coded bins exceeds the remBinsPass1, there is not the syntax element being encoded/decoded prior to the coding of the syntax element dec_abs_level for the level value of the transform coefficient. Accordingly, in case of (ii), since there is not the syntax element being encoded/decoded prior to the coding of the level value, an average level value is large as compared with the case of (i). Accordingly, in case of (i), the rice parameter lookup table generating a codeword that is advantageous to the small level value as in Table 14 may be allocated, whereas in case of (ii), the rice parameter lookup table generating a codeword that is advantageous to the large level value as in Table 15 may be allocated. As described above, in case of using the remBinsPass1 value that is already given information in determining the rice parameter lookup table, encoding/decoding of an additional flag or syntax element to know the table information is unnecessary.

As another example, if the number of encoded/decoded context coded bins in the residual data coding for the transform skip mode exceeds the MaxCcbs, the existing syntax configuration can be simplified. Some of syntax elements sig_coeff_flag, coeff_sign_flag, abs_level_gtx_flag[0], par_level_flag, abs_level_gtx_flag[1], abs_level_gtx_flag[2], abs_level_gtx_flag[3], and abs_level_gtx_flag[4] being encoded/decoded in the existing transform skip residual data coding may be omitted or may not be encoded/decoded for the coding performance or complexity reduction effect. Even in this case, since the average level values differ depending on the existing syntax configuration and the simplified syntax configuration in the same manner, it may provide better coding performance to determine the rice parameter lookup table using the MaxCcbs information and/or coefficient location information. As described above, by using the MaxCcbs and the coefficient location information, being the already known information, the encoding/decoding of the additional flag or syntax to know the table information is unnecessary in determining the rice parameter lookup table.

As another example, the rice parameter lookup table may be determined using the lossless or near-lossless coding without encoding/decoding the additional flag or syntax. The lossless or near-lossless coding may be performed in the unit of a picture, slice, CU block, or TU block, and the level value of the residual data is generally large as compared with a lossy coding. Accordingly, the decoding apparatus may use whether the current picture, slice, CU block, or TU block has been coded losslessly or near-losslessly in determining the rice parameter lookup table, and in this case, the encoding/decoding of the additional flag or syntax is also unnecessary. As an example, in case that the lossless or near-lossless coding is not performed, the decoding apparatus may derive the rice parameter using the rice parameter lookup table having relatively small minimum value/maximum value of the rice parameter as in Table 14, whereas in case that the lossless or near-lossless coding is performed, the decoding apparatus may derive the rice parameter using the rice parameter lookup table having relatively large minimum value/maximum value of the rice parameter as in Table 15.

As another embodiment, in case that the block unit variable quantization is used, the rice parameter lookup table may be selected using quantization coefficient information. In general, large level values occur frequently in case of a low quantization coefficient, whereas small level values occur frequently in case of a high quantization coefficient. Also, average level values are the same as those. Accordingly, as an example, if the value of the quantization coefficient is equal to or larger than a threshold value, the decoding apparatus may derive the rice parameter using the rice parameter lookup table having the relatively small minimum value/maximum value of the rice parameter as in Table 14, whereas if the value of the quantization coefficient is smaller than the threshold value, the decoding apparatus may derive the rice parameter using the rice parameter lookup table having the relatively large minimum value/maximum value of the rice parameter as in Table 15.

Meanwhile, as another embodiment, a method may be used, in which one rice parameter lookup table is used for the level coding, but the same effect as that using the plurality of rice parameter lookup tables is obtained. As an example, the encoding apparatus and the decoding apparatus may determine the index of the rice parameter lookup table based on whether a specific condition is satisfied. Here, whether the specific condition is satisfied may be determined based on the configuration of the syntax (the existence/nonexistence or the type of the syntax element) being coded prior to the syntax element representing the level value of the transform coefficient, whether the number of encoded/decoded context coded bins in the residual data coding exceeds the maximum number of available context coded bins configured in the context coded bin constraint algorithm, whether lossless or near-lossless coding is applied, or whether the sum locSumAbs of the level value(s) of the neighboring coefficient(s) of the current transform coefficient is larger than the threshold value or the table size (maximum value of locSumAbs of the rice parameter lookup table). The above-described specific condition may be applied even to the embodiment in which the plurality of rice parameter lookup tables are used. That is, whether the above-described specific condition is satisfied may be used in selecting any one of the plurality of rice parameter lookup tables in order to derive the rice parameter for the transform coefficient.

In the embodiment in which one rice parameter lookup table is used to derive the rice parameter for the transform coefficient, the value of the rice parameter, as an example, may be determined as in the following equations in case that the specific condition is satisfied.

$$\text{Rice parameter} = \textit{RiceParamTable}[\text{ index }] \quad \text{[Equation 15]}$$

$$\text{Rice parameter} = \textit{RiceParamTable}[\text{ index} + \text{shift }] + \text{offset} \quad \text{[Equation 16]}$$

Here, RiceParamTable means a rice parameter lookup table. Further, index represents an index value of the rice parameter lookup table. As an example, the index value may be determined based on locSumAbs being derived by the pseudo code of Table 10. Alternatively, the index value may be determined based on an index selection method configured in the standard.

Referring to Equation 15 and Equation 16, if the specific condition is not satisfied, the encoding apparatus and the decoding apparatus may read the rice parameter corresponding to the index value from the rice parameter lookup table

configured by default, and may derive the value thereof. If the specific condition is satisfied, a shift and/or an offset may be used. Here, the shift and/or the offset may have 0, a positive number, or a negative number. If the shift is a positive number, a larger rice parameter value than the rice parameter selected by the index may be derived from the rice parameter lookup table. If the shift is a negative number, a smaller rice parameter value than the rice parameter selected by the index may be derived from the rice parameter lookup table.

For example, in the present embodiment, a rice parameter lookup table as in the following Table 16 may be used.

TABLE 16

	index															
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
cRiceParam	0	0	0	0	1	1	1	1	1	1	1	2	2	2	2	2
	index															
	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
cRiceParam	2	2	2	2	2	2	3	3	3	3	3	3	3	3	3	3

Referring to Table 16, if the specific condition is not satisfied, and the index is 3, 0 is derived as the rice parameter value. If the specific condition is satisfied, the index is 3, and the shift is 1 (offset is 0), 1 is derived as the rice parameter value although the index is 3.

Meanwhile, if the offset is a positive number, a larger rice parameter value than the maximum rice parameter value of the rice parameter lookup table may be derived.

For example, if the index is 31 and the specific condition is satisfied in case that the rice parameter lookup table of Table 16 is used, 3 is derived as the rice parameter value. If the specific condition is satisfied and the offset is 2 (shift is 0), 5 is derived as the rice parameter value although the index is 31. That is, in case that the offset is used, a larger rice parameter value than the maximum rice parameter value defined in the rice parameter lookup table may be derived. Accordingly, through the rice parameter lookup table of Table 16, only the zeroth to third rice parameters can be derived, but in case that the offset is used, the usable range of the rice parameter may be increased to the zeroth to fifth rice parameters.

Through the above-described maximum rice parameter expansion, the binarization can be efficiently performed up to a high level value. Accordingly, in the present embodiment, in not only a general coding environment but also a high bit-rate environment (in which a low quantization parameter is used), and in a near-lossless or lossless environment, an advantage of coding can be taken. Further, according to the present embodiment, transmission of an additional rice parameter lookup table and/or an additional syntax is not necessary.

FIGS. **6** and **7** schematically illustrate an example of an entropy encoding method and related components according to an embodiment of the present document.

A rice parameter deriving method disclosed in FIG. **6** may be performed by the encoding apparatus **200** disclosed in FIG. **2** and FIG. **7**. Specifically, for example, S**600** to S**620** of FIG. **6** may be performed by the rice parameter deriver **241** of the entropy encoder **240**. S**630** of FIG. **6** may be performed by the binarizer **242** of the entropy encoder **240**, and S**640** of FIG. **6** may be performed by the entropy encoding processor **243** of the entropy encoder **240**.

The entropy encoding method disclosed in FIG. **6** may include the above-described embodiments in the present document.

Referring to FIG. **6** and FIG. **7**, the entropy encoder **240** performs a residual coding procedure for (quantized) transform coefficients. Here, the transform coefficients may be exchangeably used with residual coefficients. The entropy encoder **240** may perform residual coding of the (quantized) transform coefficients in the current block (current CB or current TB) in accordance with the scan order. For example, the entropy encoder **240** may generate and encode various syntax elements for the residual information as indicated in Table 1 to Table 4. As an example, the rice parameter deriver **241** of the entropy encoder **240** may generate (or derive) information representing the level value of the current transform coefficient (quantized transform coefficient or current (quantized) residual coefficient) in the current block (S**600**). Here, the information representing the level value of the current transform coefficient may include at least one of abs_remainder[n] or dec_abs_level[n]. The value of the abs_remainder[n] may be derived based on values of sig_coeff_flag[xC][yC], abs_level_gtx_flag[n][0], par_level_flag[n], and abs_level_gtx_flag[n][1]. The value of the dec_abs_level[n] may be derived as the level value of the transform coefficient. In case that the number of predetermined context coded bins in the corresponding block (CU or TU) reaches a predetermined threshold value in accordance with the scan order, the encoding apparatus may code the level value of the subsequent transform coefficient based on the dec_abs_level[n].

The rice parameter deriver **241** of the entropy encoder **240** may configure a plurality of rice parameter lookup tables, and may select one of the tables for the information representing the level value of the transform coefficient (S**610**). For example, the rice parameter deriver **241** of the entropy encoder **240** may select the rice parameter lookup table being used for the level coding of the current transform coefficient (or coefficient group, transform block, or coding block) among the plurality of rice parameter lookup tables by using already given information, such as the configuration of the syntax element (the existence/nonexistence or the type of the syntax element) being coded prior to the syntax element representing the level value of the current transform coefficient, whether the number of encoded/decoded context coded bins in the residual data coding exceeds the maximum number of available context coded bins configured in the context coded bin constraint algorithm, whether lossless or near-lossless coding is performed, or quantization coefficient information. Selection information on the corresponding rice parameter lookup table may be signaled implicitly or explicitly. For example, the selection information may

correspond to the syntax element for transmitting information representing the rice parameter lookup table selected by the encoding apparatus.

The rice parameter deriver **241** of the entropy encoder **240** may derive the rice parameter for the information (abs_remainder[n] or dec_abs_level[n]) representing the level value of the current transform coefficient based on the selected rice parameter lookup table and the neighboring (or reference) transform coefficient of the current transform coefficient (S**620**). Specifically, the rice parameter deriver **241** of the entropy encoder **240** may derive the rice parameter for (the coefficient of) the current scanning location by using the selected rice parameter lookup table based on the above-described locSumAbs. The locSumAbs may be derived based on AbsLevel and/or sig_coeff_flag of the neighboring transform coefficients. It is apparent to those skilled in the art that the procedure of deriving the rice parameter may be omitted with respect to sig_coeff_flag, par_level_flag, and abs_level_gtx_flag, being binarized with a fixed length without using the rice parameter. With respect to the sig_coeff_flag, par_level_flag, and abs_level_gtx_flag, another type binarization, which is not the binarization based on the rice parameter, may be performed.

The binarizer **242** of the entropy encoder **240** may derive a bin string for the information (abs_remainder[n] or dec_abs_level[n]) representing the level value of the transform coefficient by performing binarization based on the derived rice parameter (S**630**). The length of the bin string may be adaptively determined by the derived rice parameter.

The entropy encoding processor **243** of the entropy encoder **240** may perform entropy encoding based on the bin string for the information (abs_remainder[n] or dec_abs_level[n]) representing the level value of the transform coefficient (S**640**). The entropy encoding processor **243** of the entropy encoder **240** may perform context-based entropy encoding of the bin string based on a context-adaptive arithmetic coding (CABAC) entropy coding technique, and its output may be included in the bitstream. In this case, the encoding apparatus may perform entropy coding by deriving context information by bins of the bin string, and may update the context information by bins. As described above, the bitstream may include various kinds of information for image/video decoding, such as prediction information, in addition to residual information including information on the abs_remainder[n] or dec_abs_level[n]. The bitstream may further include the selection information on the rice parameter table. The bitstream may be transferred to the decoding apparatus through a (digital) storage medium or a network.

FIG. **8** schematically illustrates an example of an entropy encoding method according to another embodiment of the present document.

A rice parameter deriving method disclosed in FIG. **8** may be performed by the encoding apparatus **200** disclosed in FIG. **2** and FIG. **7**. Specifically, for example, S**800** to S**820** of FIG. **8** may be performed by the rice parameter deriver **241** of the entropy encoder **240**. S**830** of FIG. **8** may be performed by the binarizer **242** of the entropy encoder **240**, and S**840** of FIG. **8** may be performed by the entropy encoding processor **243** of the entropy encoder **240**.

The entropy encoding method disclosed in FIG. **8** may include the above-described embodiments in the present document.

Referring to FIG. **7** and FIG. **8**, the entropy encoder **240** performs a residual coding procedure for (quantized) transform coefficients. Here, the transform coefficients may be exchangeably used with residual coefficients. The entropy encoder **240** may perform residual coding of the (quantized) transform coefficients in the current block (current CB or current TB) in accordance with the scan order. For example, the entropy encoder **240** may generate and encode various syntax elements for the residual information as indicated in Table 1 to Table 4. As an example, the rice parameter deriver **241** of the entropy encoder **240** may generate (or derive) information representing the level value of the current transform coefficient (quantized transform coefficient or current (quantized) residual coefficient) in the current block (S**800**). Here, the information representing the level value of the current transform coefficient may include at least one of abs_remainder[n] or dec_abs_level[n]. The value of the abs_remainder[n] may be derived based on values of sig_coeff_flag[xC][yC], abs_level_gtx_flag[n][0], par_level_flag[n], and abs_level_gtx_flag[n][1]. The value of the dec_abs_level[n] may be derived as the level value of the transform coefficient. In case that the number of predetermined context coded bins in the corresponding block (CU or TU) reaches a predetermined threshold value in accordance with the scan order, the encoding apparatus may code the level value of the subsequent transform coefficient based on the dec_abs_level[n].

The rice parameter deriver **241** of the entropy encoder **240** may determine an index value of a rice lookup table for information representing the level value of the transform coefficient (S**810**). For example, the rice parameter deriver **241** of the entropy encoder **240** may determine the index value of the rice parameter lookup table based on the configuration of the syntax element (the existence/nonexistence or the type of the syntax element) being coded prior to the syntax element representing the level value of the current transform coefficient, whether the number of encoded/decoded context coded bins in the residual data coding exceeds the maximum number of available context coded bins configured in the context coded bin constraint algorithm, whether lossless or near-lossless coding is performed, quantization coefficient information (value), or whether a sum of level value(s) of neighboring coefficient(s) of the current transform coefficient is larger than a threshold value (or the size of the rice parameter lookup table), or may change the corresponding index value by adding a shift to the index value.

The rice parameter deriver **241** of the entropy encoder **240** may derive the rice parameter for the information (abs_remainder[n] or dec_abs_level[n]) representing the level value of the current transform coefficient from the rice parameter lookup table based on the index value (S**820**). For example, the rice parameter deriver **241** of the entropy encoder **240** may derive the rice parameter for (the coefficient of) the current scanning location by using Equation 15 and/or Equation 16 as described above based on the index value. In case that Equation 16 is used, the value of the rice parameter may be changed to a value obtained by adding an offset value to the index value. The index value may be derived based on the above-described locSumAbs, or may be determined based on the index selection method configured in the standard. The locSumAbs may be derived based on the AbsLevel and/or the sig_coeff_flag of the neighboring transform coefficients.

It is apparent to those skilled in the art that the procedure of deriving the rice parameter may be omitted with respect to sig_coeff_flag, par_level_flag, and abs_level_gtx_flag, being binarized with a fixed length without using the rice parameter. With respect to the sig_coeff_flag, par_level_

flag, and abs_level_gtx_flag, another type binarization, which is not the binarization based on the rice parameter, may be performed.

The binarizer **242** of the entropy encoder **240** may derive a bin string for the information (abs_remainder[n] or dec_abs_level[n]) representing the level value of the transform coefficient by performing binarization based on the derived rice parameter (S**830**). The length of the bin string may be adaptively determined by the derived rice parameter.

The entropy encoding processor **243** of the entropy encoder **240** may perform entropy encoding based on the bin string for the information (abs_remainder[n] or dec_abs_level[n]) representing the level value of the transform coefficient (S**840**). The entropy encoding processor **243** of the entropy encoder **240** may perform context-based entropy encoding of the bin string based on a context-adaptive arithmetic coding (CABAC) entropy coding technique, and its output may be included in the bitstream. In this case, the encoding apparatus may perform entropy coding by deriving context information by bins of the bin string, and may update the context information by bins. As described above, the bitstream may include various kinds of information for image/video decoding, such as prediction information, in addition to residual information including information on the abs_remainder[n] or dec_abs_level[n]. The bitstream may further include the selection information on the rice parameter table. The bitstream may be transferred to the decoding apparatus through a (digital) storage medium or a network.

FIGS. **9** and **10** schematically illustrate an example of an entropy decoding method and related components according to an embodiment of the present document.

A rice parameter deriving method disclosed in FIG. **9** may be performed by the decoding apparatus **300** disclosed in FIG. **3** and FIG. **10**. Specifically, for example, S**900** to S**920** of FIG. **9** may be performed by the rice parameter deriver **311** of the entropy decoder **310**. S**930** of FIG. **9** may be performed by the binarizer **312** of the entropy decoder **310**, and S**940** of FIG. **9** may be performed by the entropy decoding processor **313** of the entropy decoder **310**.

The entropy decoding method disclosed in FIG. **9** may include the above-described embodiments in the present document.

Referring to FIGS. **9** and **10**, the entropy decoder may derive (quantized) transform coefficients by decoding encoded residual information. Here, the transform coefficients may be interchangeably used with residual coefficients. The decoding apparatus may derive (quantized) transform coefficients by decoding encoded residual information for the current block (current CB or current TB). For example, the decoding apparatus may decode various syntax elements about the residual information as indicated in Table 1 to Table 4, interpret values of related syntax elements, and derive the (quantized) transform coefficients based on this.

Specifically, the rice parameter deriver **311** of the entropy decoder **310** may obtain information (abs_remainder[n] or dec_abs_level[n]) representing the level value of the current transform coefficient (quantized transform coefficient or current (quantized) residual coefficient) from a bitstream (S**900**). Further, the rice parameter lookup table for the information representing the level value among the plurality of rice parameter lookup tables may be selected (S**910**).

For example, the rice parameter deriver **311** of the entropy decoder **310** may configure a plurality of rice parameter lookup tables, and may select one of the tables. For this, selection information for selecting one of the tables may be explicitly signaled. The selection information may correspond to the syntax element for transmitting information representing the rice parameter lookup table selected by the encoding apparatus. In this case, the rice parameter deriver **311** of the entropy decoder **310** may obtain the syntax element representing the information on the rice parameter lookup table from the bitstream, and based on this, may select any one of the plurality of rice parameter lookup tables.

Further, the rice parameter deriver **311** of the entropy decoder **310** may select the rice parameter lookup table being used for the level coding of the current transform coefficient (or coefficient group, transform block, or coding block) among the plurality of rice parameter lookup tables based on at least one of already given information, such as the configuration of the syntax element (the existence/nonexistence or the type of the syntax element) being coded prior to the syntax element representing the level value of the current transform coefficient, information on the maximum number of available context coded bins configured in the context coded bin constraint algorithm, whether the current block is coded losslessly or near-losslessly, or quantization coefficient information for the current transform coefficient. Here, the syntax element being coded prior to the syntax element representing the level value of the current transform coefficient may include at least one of sig_coeff_flag, abs_level_gtx_flag, par_level_flag, or coeff_sign_flag, and the rice parameter lookup table may be selected based on whether at least one of them is decoded.

The rice parameter deriver **311** of the entropy decoder **310** may derive the rice parameter for the information (abs_remainder[n] or dec_abs_level[n]) representing the level value of the current transform coefficient based on the selected rice parameter lookup table and the neighboring (or reference) transform coefficient of the current transform coefficient (S**920**). Specifically, the rice parameter deriver **311** of the entropy decoder **310** may derive the rice parameter for (the coefficient of) the current scanning location by using the selected rice parameter lookup table based on the above-described locSumAbs. The locSumAbs may be derived based on AbsLevel and/or sig_coeff_flag of the neighboring transform coefficients. It is apparent to those skilled in the art that the procedure of deriving the rice parameter may be omitted with respect to sig_coeff_flag, par_level_flag, and abs_level_gtx_flag, being binarized with a fixed length without using the rice parameter. With respect to the sig_coeff_flag, par_level_flag, and abs_level_gtx_flag, another type binarization, which is not the binarization based on the rice parameter, may be performed.

The binarizer **312** of the entropy decoder **310** may derive a bin string for the information (abs_remainder[n] or dec_abs_level[n]) representing the level value of the transform coefficient by performing binarization based on the derived rice parameter (S**930**). As an example, the binarizer **312** of the entropy decoder **310** may derive available bin strings for available values of the abs_remainder[n] or the dec_abs_level[n] through the binarization procedure. The length of the available bin string may be adaptively determined by the derived rice parameter.

The entropy decoding processor **313** of the entropy decoder **310** may derive the level value of the transform coefficient by performing entropy decoding based on the bin string for the information (abs_remainder[n] or dec_abs_level[n]) representing the level value of the transform coefficient (S**940**). For example, the entropy decoding processor **313** of the entropy decoder **310** may compare the derived bin string with the available bin strings while sequentially parsing and decoding the bins/bits for the

abs_remainder[n] or dec_abs_level[n]. If the derived bin string is equal to one of the available bin strings, the value corresponding to the corresponding bin string may be derived as the value of the abs_remainder[n]. Otherwise, the next bit in the bitstream may be further parsed and decoded, and then the comparison procedure may be performed. Through the above process, the corresponding information may be signaled using a variable length bit even without using a start bit or an end bit for specific information (specific syntax element) in the bitstream. Through this, a relatively smaller number of bits may be allocated to a small value, and thus the overall coding efficiency can be enhanced.

The decoding apparatus may perform context-based entropy decoding of respective bins in the bin string from the bitstream based on the CABAC entropy coding technique. In this case, the decoding apparatus may perform entropy coding by deriving context information by bins of the bin string, and may update the context information by bins. The entropy decoding procedure may be performed by the entropy decoding processor **313** in the entropy decoder **310**. As described above, the bitstream may include various kinds of information for image/video decoding, such as prediction information, in addition to residual information including information on the abs_remainder[n] or the dec_abs_level [n]. The bitstream may further include selection information on the rice parameter lookup table. As described above, the bitstream may be transferred to the decoding apparatus through a (digital) storage medium or a network.

The decoding apparatus may derive (quantized) transform/residual coefficients based on the entropy decoding, and based on this, may derive residual samples for the current block by performing dequantization and/or inverse transform procedures as needed. Reconstructed samples may be generated based on the residual samples and prediction samples derived through inter prediction and/or intra prediction, and a reconstructed block/picture including the reconstructed samples may be generated.

FIG. **11** schematically illustrates an example of an entropy decoding method according to another embodiment of the present document.

A rice parameter deriving method disclosed in FIG. **11** may be performed by the decoding apparatus **300** disclosed in FIG. **3** and FIG. **10**. Specifically, for example, S**1100** to S**1120** of FIG. **11** may be performed by the rice parameter deriver **311** of the entropy decoder **310**. S**1130** of FIG. **11** may be performed by the binarizer **312** of the entropy decoder **310**, and S**1140** of FIG. **11** may be performed by the entropy decoding processor **313** of the entropy decoder **310**.

The entropy decoding method disclosed in FIG. **11** may include the above-described embodiments in the present document.

Referring to FIGS. **10** and **11**, the entropy decoder may derive (quantized) transform coefficients by decoding encoded residual information. Here, the transform coefficients may be interchangeably used with residual coefficients. The decoding apparatus may derive (quantized) transform coefficients by decoding encoded residual information for the current block (current CB or current TB). For example, the decoding apparatus may decode various syntax elements about the residual information as indicated in Table 1 to Table 4, interpret values of related syntax elements, and derive the (quantized) transform coefficients based on this.

Specifically, the rice parameter deriver **311** of the entropy decoder **310** may obtain information (abs_remainder[n] or dec_abs_level[n]) representing the level value of the current transform coefficient (quantized transform coefficient or current (quantized) residual coefficient) from a bitstream (S**1100**). Further, an index value of a rice parameter lookup table for the information representing the level value may be determined (S**1110**). For example, the rice parameter deriver **311** of the entropy decoder **310** may determine and/or change the index value of the rice parameter lookup table based on at least one of the configuration of the syntax element (the existence/nonexistence or the type of the syntax element) being coded prior to the information representing the level value of the current transform coefficient, the maximum number of available context coded bins configured in the context coded bin constraint algorithm, whether lossless or near-lossless coding is performed, quantization coefficient information (value) for the transform coefficient, or level value(s) of neighboring coefficient(s) of the current transform coefficient.

For example, the rice parameter deriver **311** of the entropy decoder **310** may determine the index value based on whether at least one syntax element sig_coeff_flag, abs_level_gtx_flag, par_level_flag, or coeff_sign_flag is decoded from the bitstream, or may change the index value by adding a shift to the index value. Alternatively, the index value may be determined based on whether the sum of level values of the neighboring transform coefficients of the transform coefficient is equal to or larger than a threshold value, or the index value may be changed by adding the shift to the index value.

The rice parameter deriver **311** of the entropy decoder **310** may derive the rice parameter for the information (abs_remainder[n] or dec_abs_level[n]) representing the level value of the current transform coefficient from the rice parameter lookup table based on the index value (S**1120**). For example, the rice parameter deriver **311** of the entropy decoder **310** may derive the rice parameter for (the coefficient of) the current scanning location by using Equation 15 and/or Equation 16 as described above based on the index value. In case that Equation 16 is used, the rice parameter value may be changed to a value obtained by adding an offset value to the index value. The index value may be derived based on the locSumAbs as described above, or may be determined based on the index selection method configured in the standard.

It is apparent to those skilled in the art that the procedure of deriving the rice parameter may be omitted with respect to sig_coeff_flag, par_level_flag, and abs_level_gtx_flag, being binarized with a fixed length without using the rice parameter. With respect to the sig_coeff_flag, par_level_flag, and abs_level_gtx_flag, another type binarization, which is not the binarization based on the rice parameter, may be performed.

The binarizer **312** of the entropy decoder **310** may derive a bin string for the information (abs_remainder[n] or dec_abs_level[n]) representing the level value of the transform coefficient by performing binarization based on the derived rice parameter (S**1130**). As an example, the binarizer **312** of the entropy decoder **310** may derive available bin strings for available values of the abs_remainder[n] or the dec_abs_level[n] through the binarization procedure. The length of the available bin string may be adaptively determined by the derived rice parameter.

The entropy decoding processor **313** of the entropy decoder **310** may derive the level value of the transform coefficient by performing entropy decoding based on the bin string for the information (abs_remainder[n] or dec_abs_level[n]) representing the level value of the transform coefficient (S**1140**). For example, the entropy decoding processor **313** of the entropy decoder **310** may compare the

derived bin string with the available bin strings while sequentially parsing and decoding the bins/bits for the abs_remainder[n] or dec_abs_level[n]. If the derived bin string is equal to one of the available bin strings, the value corresponding to the corresponding bin string may be derived as the value of the abs_remainder[n]. Otherwise, the next bit in the bitstream may be further parsed and decoded, and then the comparison procedure may be performed. Through the above process, the corresponding information may be signaled using a variable length bit even without using a start bit or an end bit for specific information (specific syntax element) in the bitstream. Through this, a relatively smaller number of bits may be allocated to a small value, and thus the overall coding efficiency can be enhanced.

The decoding apparatus may perform context-based entropy decoding of respective bins in the bin string from the bitstream based on the CABAC entropy coding technique. In this case, the decoding apparatus may perform entropy coding by deriving context information by bins of the bin string, and may update the context information by bins. The entropy decoding procedure may be performed by the entropy decoding processor **313** in the entropy decoder **310**. As described above, the bitstream may include various kinds of information for image/video decoding, such as prediction information, in addition to residual information including information on the abs_remainder[n] or the dec_abs_level [n]. The bitstream may further include selection information on the rice parameter lookup table. As described above, the bitstream may be transferred to the decoding apparatus through a (digital) storage medium or a network.

The decoding apparatus may derive (quantized) transform/residual coefficients based on the entropy decoding, and as needed, may derive residual samples for the current block by performing dequantization and/or inverse transform procedures based on this. Reconstructed samples may be generated based on the residual samples and prediction samples derived through inter prediction and/or intra prediction, and a reconstructed block/picture including the reconstructed samples may be generated.

FIG. **12** illustrates a video/image encoding method according to an embodiment of the present document.

The video/image encoding method disclosed in FIG. **12** may be performed by the encoding apparatus **200** disclosed in FIG. **2**. Specifically, for example, S**1200** of FIG. **12** may be performed by the predictor **220** of the encoding apparatus, and S**1210** may be performed by the subtractor **231** of the encoding apparatus. S**1220** may be performed by the transformer **232** of the encoding apparatus, S**1230** may be performed by the quantizer **233** of the encoding apparatus, and S**1240** may be performed by the entropy encoder **240** of the encoding apparatus. S**800** to S**830** as described above with reference to FIG. **8** may be included in the procedure of S**1240**.

Referring to FIG. **12**, the encoding apparatus may derive prediction samples through prediction for the current block (S**1200**). The encoding apparatus may determine whether to perform inter prediction or intra prediction with respect to the current block, and may determine a specific inter prediction mode or a specific intra prediction mode based on an RD cost. In accordance with the determined mode, the encoding apparatus may derive the prediction samples for the current block.

The encoding apparatus may derive residual samples through comparison of the original samples for the current block with the prediction samples (S**1210**).

The encoding apparatus may derive transform coefficients through a transform procedure for the residual samples (S**1220**), and may derive quantized transform coefficients through quantization of the derived transform coefficients (S**1230**).

The encoding apparatus may encode image information including prediction information and residual information, and may output the encoded image information in the form of a bitstream (S**1240**). The prediction information may include information (e.g., in case that the inter prediction is applied) on prediction mode information and motion information as information related to the prediction procedure. The residual information is information about the quantized transform coefficients, and for example, may include the information disclosed in Table 1 to Table 4 described above.

The output bitstream may be transferred to the decoding apparatus through a storage medium or a network.

FIG. **13** illustrates a video/image decoding method according to an embodiment of the present document.

The video/image decoding method disclosed in FIG. **13** may be performed by the decoding apparatus **300** disclosed in FIG. **3**. Specifically, for example, S**1300** of FIG. **13** may be performed by the predictor **330** of the decoding apparatus. A procedure of deriving values of related syntax elements through decoding of prediction information included in a bitstream in S**1300** may be performed by the entropy decoder **310** of the decoding apparatus. S**1310**, S**1320**, S**1330**, and S**1340** may be performed by the entropy decoder **310**, the dequantizer **321**, the inverse transformer **322**, and the adder **340** of the decoding apparatus, respectively. S**1000** to S**1030** as described above in FIG. **10** may be included in the procedure of S**1310**.

The decoding apparatus may perform an operation corresponding to the operation performed by the encoding apparatus. The decoding apparatus may perform inter prediction or intra prediction for the current block based on received prediction information, and may derive prediction samples (S**1300**).

The decoding apparatus may derive quantized transform coefficients for the current block based on received residual information (S**1310**).

The decoding apparatus may derive transform coefficients through dequantization of the quantized transform coefficients (S**1320**).

The decoding apparatus may derive residual samples through an inverse transform procedure for the transform coefficients (S**1330**).

The decoding apparatus may generate reconstructed samples for the current block based on the prediction samples and the residual samples, and may generate a reconstructed picture based on this (S**1340**). Thereafter, as described above, an in-loop filtering procedure may be further applied to the reconstructed picture.

Although methods have been described on the basis of a flowchart in which steps or blocks are listed in sequence in the above-described embodiments, the steps of the present disclosure are not limited to a certain order, and a certain step may be performed in a different step or in a different order or concurrently with respect to that described above. Further, it will be understood by those ordinary skilled in the art that the steps of the flowcharts are not exclusive, and another step may be included therein or one or more steps in the flowchart may be deleted without exerting an influence on the scope of the present disclosure.

The aforementioned method according to the present disclosure may be in the form of software, and the encoding apparatus and/or decoding apparatus according to the pres-

ent disclosure may be included in a device for performing image processing, for example, a TV, a computer, a smart phone, a set-top box, a display device, or the like.

When the embodiments of the present disclosure are implemented by software, the aforementioned method may be implemented by a module (process or function) which performs the aforementioned function. The module may be stored in a memory and executed by a processor. The memory may be installed inside or outside the processor and may be connected to the processor via various well-known means. The processor may include Application-Specific Integrated Circuit (ASIC), other chipsets, a logical circuit, and/or a data processing device. The memory may include a Read-Only Memory (ROM), a Random Access Memory (RAM), a flash memory, a memory card, a storage medium, and/or other storage device. In other words, the embodiments according to the present disclosure may be implemented and executed on a processor, a micro-processor, a controller, or a chip. For example, functional units illustrated in the respective figures may be implemented and executed on a computer, a processor, a microprocessor, a controller, or a chip. In this case, information on implementation (for example, information on instructions) or algorithms may be stored in a digital storage medium.

In addition, the decoding apparatus and the encoding apparatus to which the embodiment(s) of the present disclosure is applied may be included in a multimedia broadcasting transceiver, a mobile communication terminal, a home cinema video device, a digital cinema video device, a surveillance camera, a video chat device, and a real time communication device such as video communication, a mobile streaming device, a storage medium, a camcorder, a video on demand (VOD) service provider, an Over The Top (OTT) video device, an internet streaming service provider, a 3D video device, a Virtual Reality (VR) device, an Augment Reality (AR) device, an image telephone video device, a vehicle terminal (for example, a vehicle (including an autonomous vehicle) terminal, an airplane terminal, or a ship terminal), and a medical video device; and may be used to process an image signal or data. For example, the OTT video device may include a game console, a Blu-ray player, an Internet-connected TV, a home theater system, a smartphone, a tablet PC, and a Digital Video Recorder (DVR).

In addition, the processing method to which the embodiment(s) of the present disclosure is applied may be produced in the form of a program executed by a computer and may be stored in a computer-readable recording medium. Multimedia data having a data structure according to the embodiment(s) of the present disclosure may also be stored in the computer-readable recording medium. The computer readable recording medium includes all kinds of storage devices and distributed storage devices in which computer readable data is stored. The computer-readable recording medium may include, for example, a Blu-ray disc (BD), a universal serial bus (USB), a ROM, a PROM, an EPROM, an EEPROM, a RAM, a CD-ROM, a magnetic tape, a floppy disk, and an optical data storage device. The computer-readable recording medium also includes media embodied in the form of a carrier wave (for example, transmission over the Internet). In addition, a bitstream generated by the encoding method may be stored in the computer-readable recording medium or transmitted through a wired or wireless communication network.

In addition, the embodiment(s) of the present disclosure may be embodied as a computer program product based on a program code, and the program code may be executed on a computer according to the embodiment(s) of the present disclosure. The program code may be stored on a computer-readable carrier.

FIG. **14** represents an example of a contents streaming system to which the embodiment of the present disclosure may be applied.

Referring to FIG. **14**, the content streaming system to which the embodiments of the present disclosure is applied may generally include an encoding server, a streaming server, a web server, a media storage, a user device, and a multimedia input device.

The encoding server functions to compress to digital data the contents input from the multimedia input devices, such as the smart phone, the camera, the camcorder and the like, to generate a bitstream, and to transmit it to the streaming server. As another example, in a case in which the multimedia input device, such as, the smart phone, the camera, the camcorder or the like, directly generates a bitstream, the encoding server may be omitted.

The bitstream may be generated by an encoding method or a bitstream generation method to which the embodiments of the present disclosure is applied. And the streaming server may temporarily store the bitstream in a process of transmitting or receiving the bitstream.

The streaming server transmits multimedia data to the user equipment on the basis of a user's request through the web server, which functions as an instrument that informs a user of what service there is. When the user requests a service which the user wants, the web server transfers the request to the streaming server, and the streaming server transmits multimedia data to the user. In this regard, the contents streaming system may include a separate control server, and in this case, the control server functions to control commands/responses between respective equipment in the content streaming system.

The streaming server may receive contents from the media storage and/or the encoding server. For example, in a case the contents are received from the encoding server, the contents may be received in real time. In this case, the streaming server may store the bitstream for a predetermined period of time to provide the streaming service smoothly.

For example, the user equipment may include a mobile phone, a smart phone, a laptop computer, a digital broadcasting terminal, a personal digital assistant (PDA), a portable multimedia player (PMP), a navigation, a slate PC, a tablet PC, an ultrabook, a wearable device (e.g., a watch-type terminal (smart watch), a glass-type terminal (smart glass), a head mounted display (HMD)), a digital TV, a desktop computer, a digital signage or the like.

Each of servers in the contents streaming system may be operated as a distributed server, and in this case, data received by each server may be processed in distributed manner.

What is claimed is:

1. A decoding apparatus for image decoding, the decoding apparatus comprising:

a memory; and

at least one processor connected to the memory, the at least one processor configured to:

receive a bitstream including information related to a level value of a transform coefficient in a current block;

determine an index value of a rice parameter lookup table for the information related to the level value of the transform coefficient;

derive a rice parameter for the information related to the level value of the transform coefficient based on the rice parameter lookup table and the index value;

derive a bin string for the information related to the level value of the transform coefficient based on the rice parameter; and

derive the level value of the transform coefficient based on the bin string,

wherein to derive the rice parameter the at least one processor further configured to:

derive a temporary rice parameter based on the index value and the rice parameter lookup table; and

derive the rice parameter by adding an offset to the temporary rice parameter.

2. The decoding apparatus of claim **1**, wherein to derive the rice parameter the at least one processor further configured to:

derive a modified index value based on the index value; and

derive the rice parameter by using the modified index value and the rice parameter lookup table,

wherein the deriving of the modified index value comprises deriving of the modified index value based on whether a sum of level values of neighboring transform coefficients of the transform coefficient is smaller than a threshold value.

3. The decoding apparatus of claim **2**, wherein the deriving of the modified index value comprises deriving of the modified index value based on the index value and a shift value.

4. The decoding apparatus of claim **1**, wherein to derive the rice parameter the at least one processor further configured to:

derive modified index value based on the index value and a shift value;

derive a temporary rice parameter based on the modified index value and the rice parameter lookup table; and

derive the rice parameter by adding an offset to the temporary rice parameter.

5. The decoding apparatus of claim **4**, wherein based on a specific syntax element which is coded prior to the information related to the level value of the transform coefficient, the modified index value is derived by using the shift value and the rice parameter is derived by using the offset.

6. The decoding apparatus of claim **5**, wherein the index value is derived based on a sum of level values of neighboring transform coefficients of the transform coefficient.

7. The decoding apparatus of claim **1**, wherein the information related to the level value of the transform coefficient includes a syntax element dec_abs level representing the level value of the transform coefficient or a syntax element abs_remainder representing a remaining level value of the transform coefficient.

8. An encoding apparatus for image encoding, the encoding apparatus comprising:

a memory; and

at least one processor connected to the memory, the at least one processor configured to:

generate information related to a level value of a transform coefficient in a current block;

determine an index value of a rice parameter lookup table for the information related to the level value of the transform coefficient;

derive a rice parameter for the information related to the level value of the transform coefficient based on the index value and the rice parameter lookup table;

derive a bin string for the information related to the level value of the transform coefficient based on the rice parameter; and

encode the bin string,

wherein to derive the rice parameter the at least one processor further configured to:

derive a temporary rice parameter based on the index value and the rice parameter lookup table; and

derive the rice parameter by adding an offset to the temporary rice parameter.

9. The encoding apparatus of claim **8**, wherein to derive the rice parameter the at least one processor further configured to:

derive a modified index value based on the index value; and

derive the rice parameter by using the modified index value and the rice parameter lookup table,

wherein the deriving of the modified index value comprises deriving of the modified index value based on whether a sum of level values of neighboring transform coefficients of the transform coefficient is smaller than a threshold value.

10. The encoding apparatus of claim **9**, wherein the deriving of the modified index value comprises deriving of the modified index value based on the index value and a shift value.

11. The encoding apparatus of claim **8**, wherein to derive the rice parameter the at least one processor further configured to:

derive modified index value based on the index value and a shift value;

derive a temporary rice parameter based on the modified index value and the rice parameter lookup table; and

derive the rice parameter by adding an offset to the temporary rice parameter.

12. The encoding apparatus of claim **11**, wherein based on a specific syntax element which is coded prior to the information related to the level value of the transform coefficient, the modified index value is derived by using the shift value and the rice parameter is derived by using the offset.

13. The encoding apparatus of claim **12**, wherein the index value is derived based on a sum of level values of neighboring transform coefficients of the transform coefficient.

14. The encoding apparatus of claim **8**, wherein the information related to the level value of the transform coefficient includes a syntax element dec_abs level representing the level value of the transform coefficient or a syntax element abs_remainder representing a remaining level value of the transform coefficient.

15. An apparatus for transmitting data for an image, the apparatus comprising:

at least one processor configured to obtain a bitstream generated by an encoding method performed by an encoding apparatus for image encoding; and

a transmitter configured to transmit the bitstream,

wherein the encoding method comprising:

generating information related to a level value of a transform coefficient in a current block;

determining an index value of a rice parameter lookup table for the information related to the level value of the transform coefficient;

deriving a rice parameter for the information related to the level value of the transform coefficient based on the index value and the rice parameter lookup table;

deriving a bin string for the information related to the level value of the transform coefficient based on the rice parameter; and

generating the bitstream by encoding the bin string,

wherein the deriving the rice parameter comprises:

deriving a temporary rice parameter based on the index value and the rice parameter lookup table; and

deriving the rice parameter by adding an offset to the temporary rice parameter.

* * * * *